Tree

Tree

- A **tree** is a non primitive and non linear data structure.

- It is a collection of data elements of same data type.

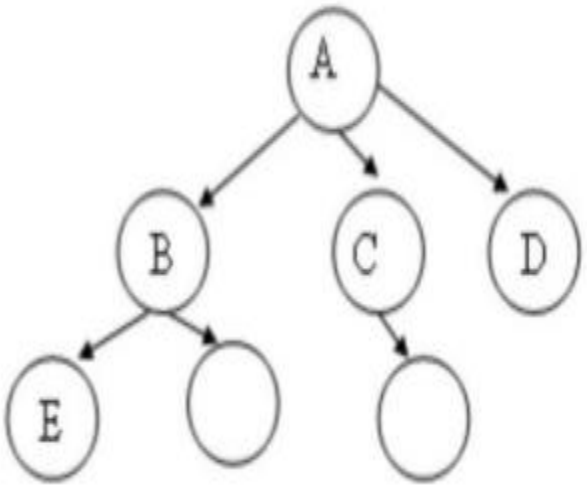
Cont…

- In a **tree** each data item is called node.

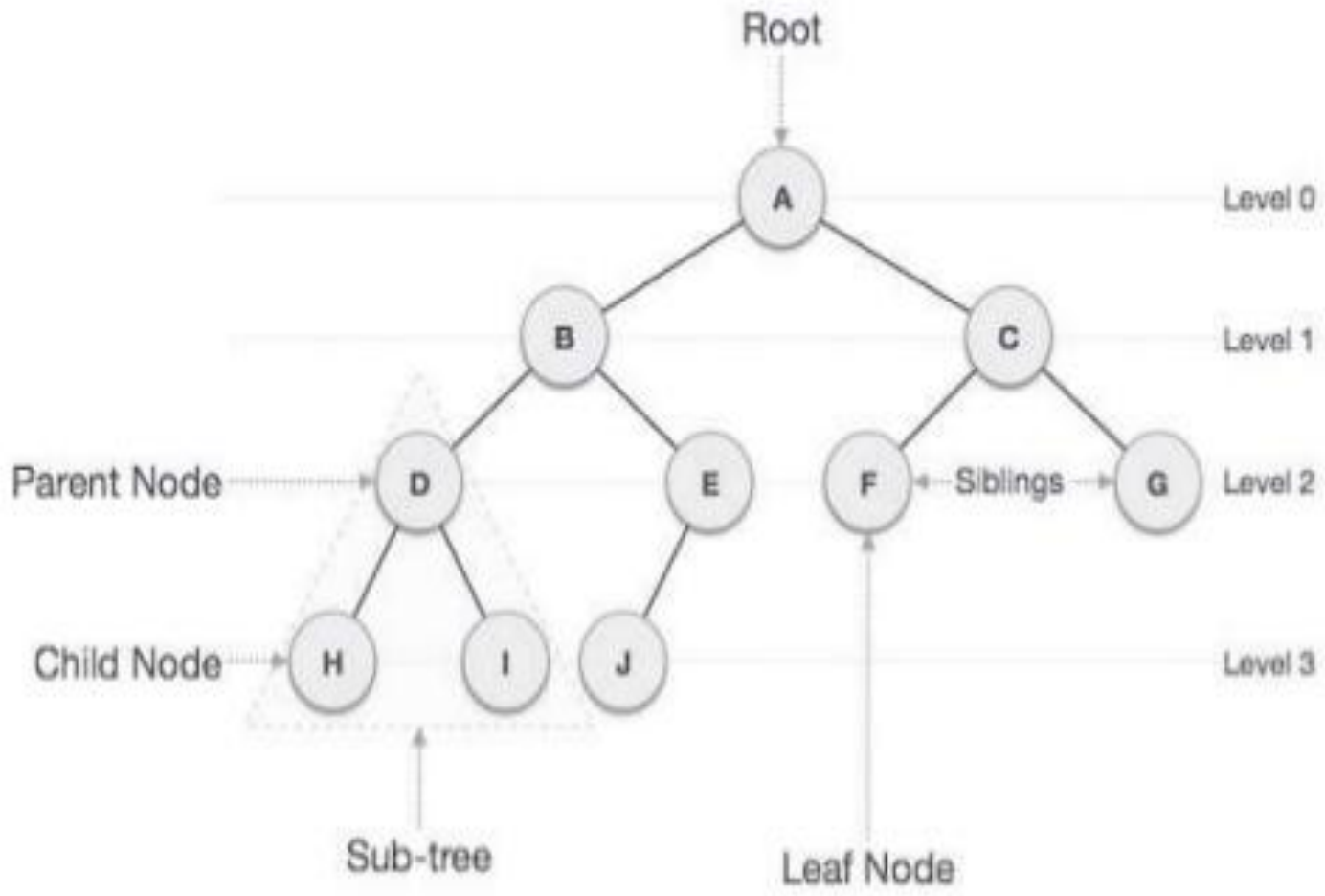
- In a **tree** each node is represented by a circle.

- **Tree** is used to represent the **hierarchical relationship** (parent-child relationship) among several **data items** (nodes).

- Tree is a sequence of nodes
- There is a starting node known as a root node
- Every node other than the root has a parent node.
- Nodes may have any number of children



A has 3 children, B, C, D
A is parent of B


Root
A
B
C
D
E
F
G
H
I
J
Level 0
Level 1
Level 2
Level 3
Parent Node
Siblings
Child Node
Sub-tree
Leaf Node

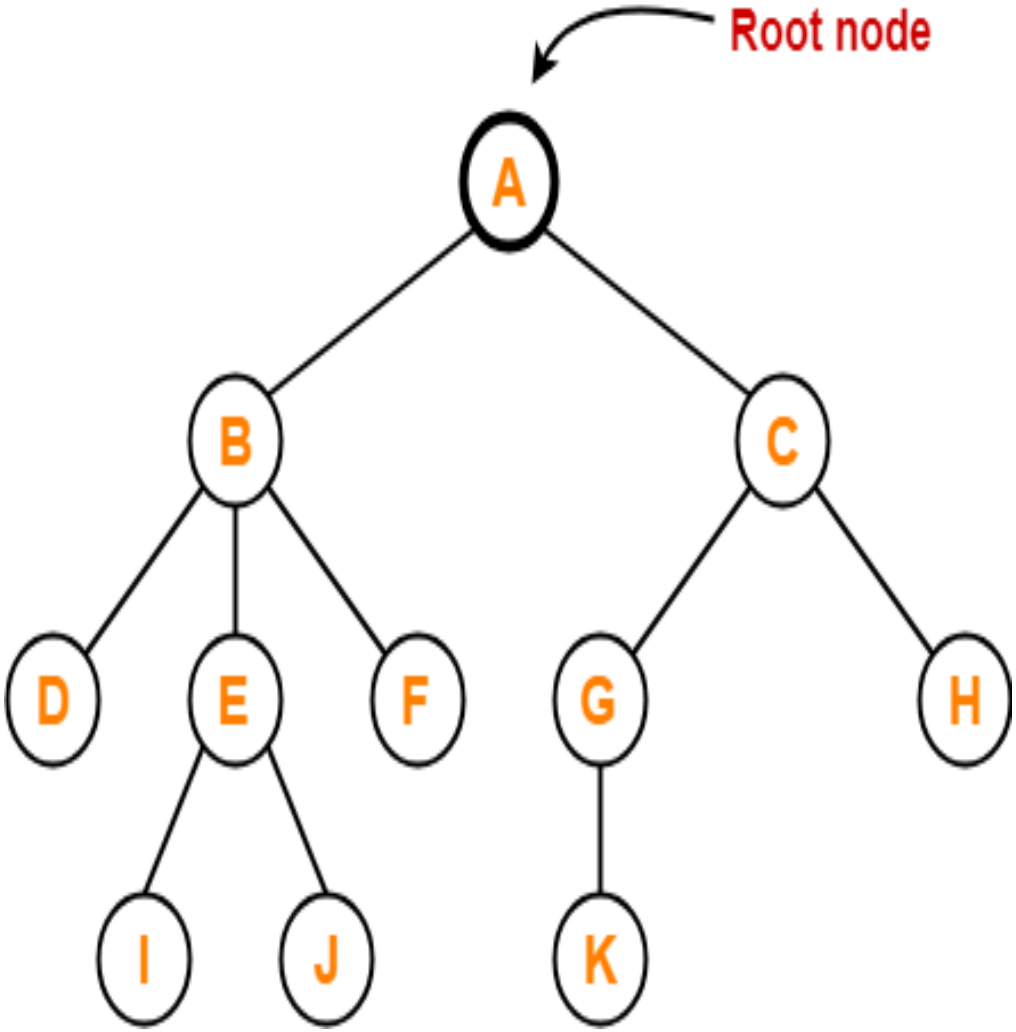
Properties of Tree

- The important properties of tree data structure are-
  - There is one and only one path between every pair of vertices **(nodes)** in a tree.
  - A tree with n vertices **(nodes)** has exactly (n-1) edges.

Tree Terminology

1. Root

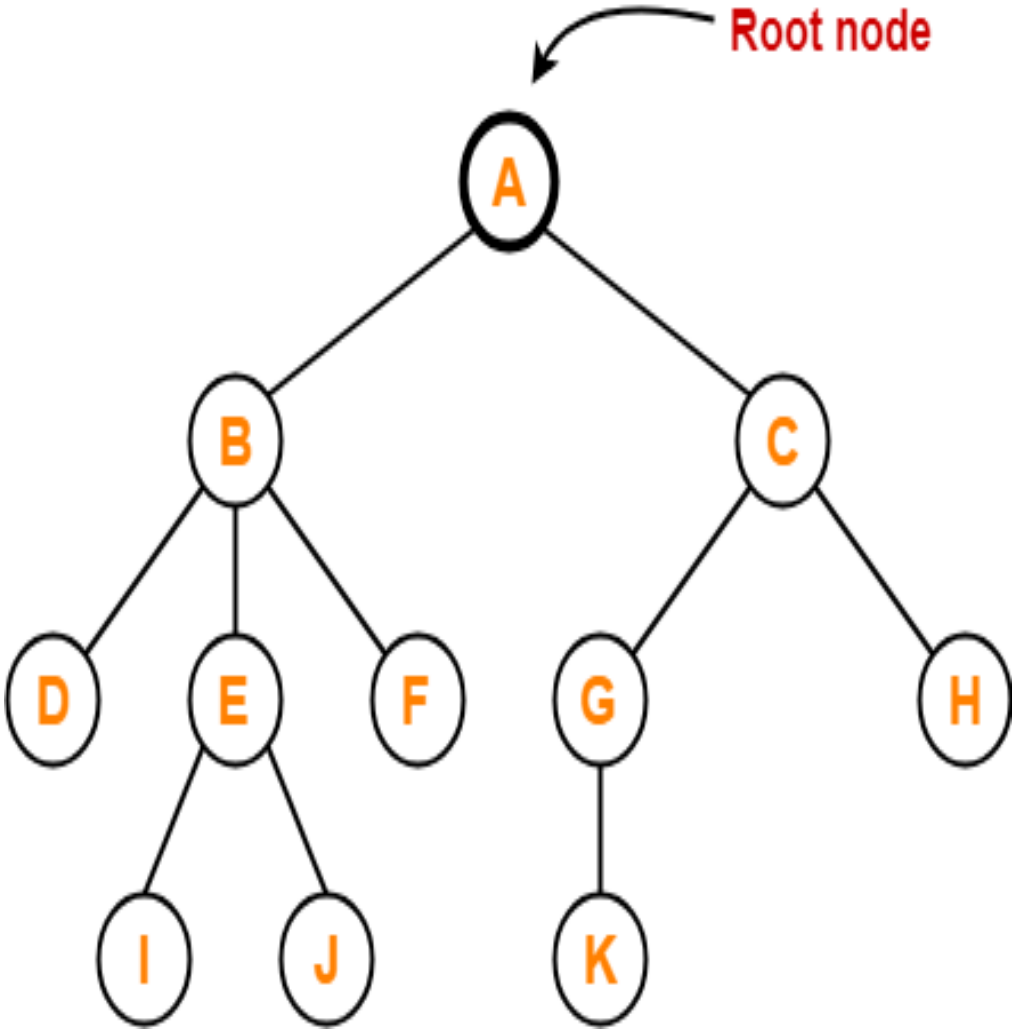
- The first node from where the tree originates is called as a **root node**.
- In any tree, there must be only one root node.
- There is no Parent node of root node.



Cont…

2. Node

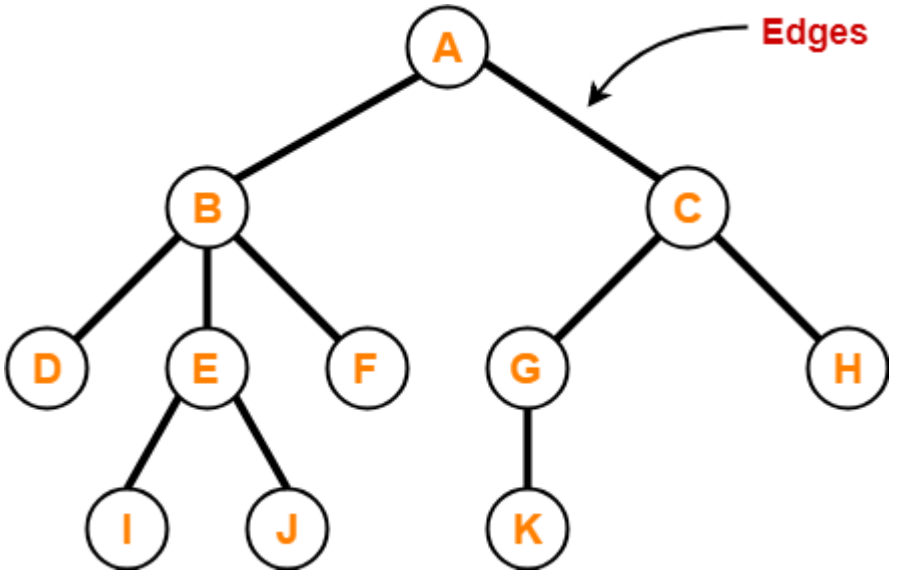
- Each data item in a tree is called a node.
- Node specifies the data information and links to other data items .
- There are 11 nodes in the following diagram



Cont…

3. Edge

- The connecting link between any two nodes is called as an **edge**.

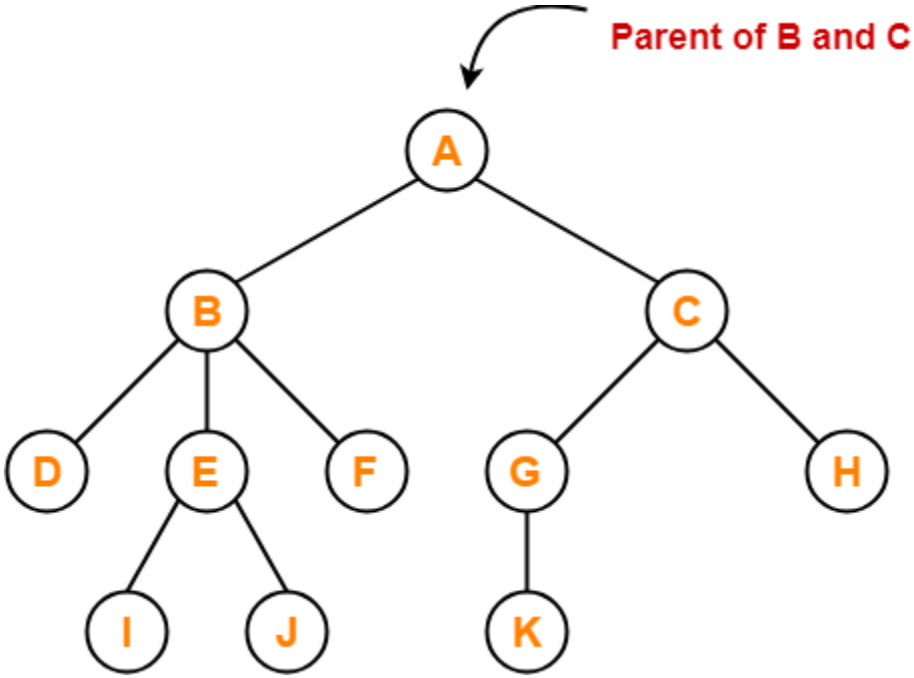


- In a tree with **n** number of **nodes**, there are exactly **(n-1)** number of **edges**.

Cont…

4. Parent

- The node which has one or more children is called as a parent node.

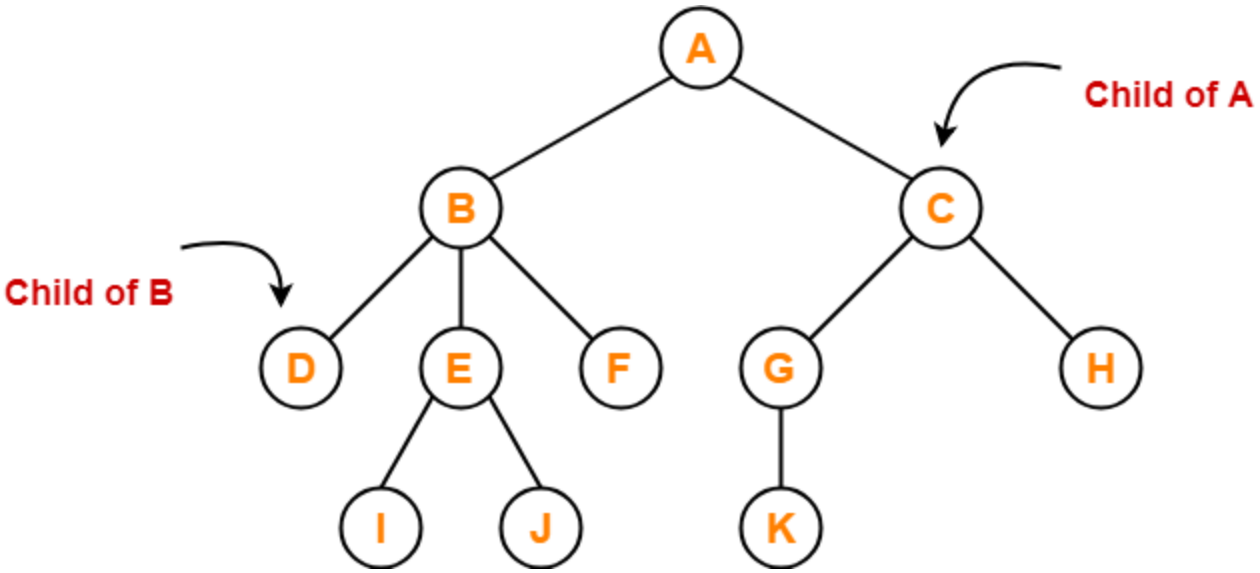


- In a tree, a parent node can have any number of child nodes.

Cont…

5. Child

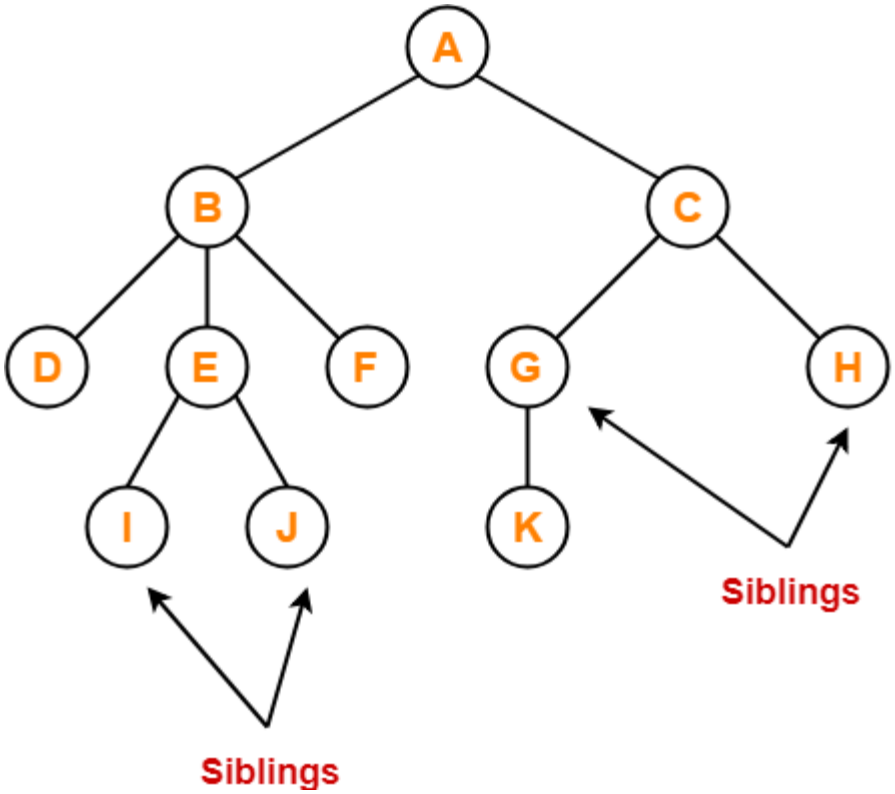
- The immediate successors of a node are called as a **child nodes**.
- All the nodes except root node are child nodes.



Cont…

6. Siblings

- Nodes which belong to the same parent are called as **siblings**.

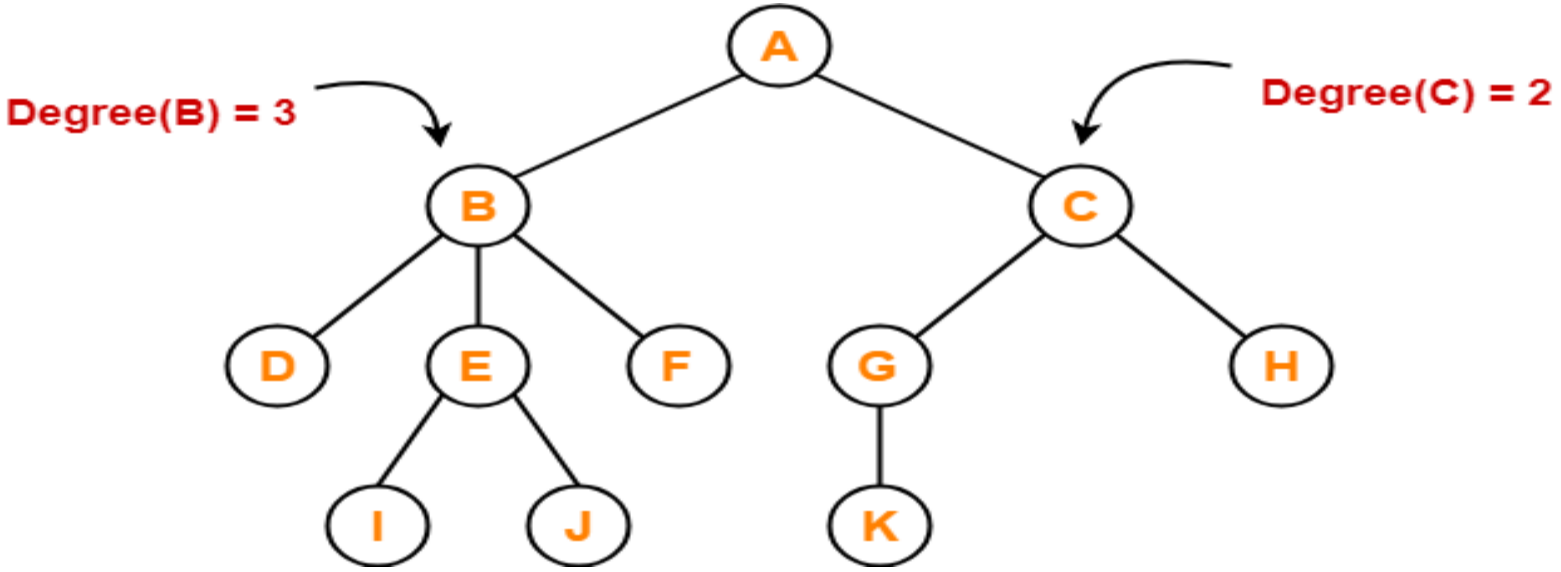


- In other words, nodes with the same parent are sibling nodes.

Cont…

7. Degree

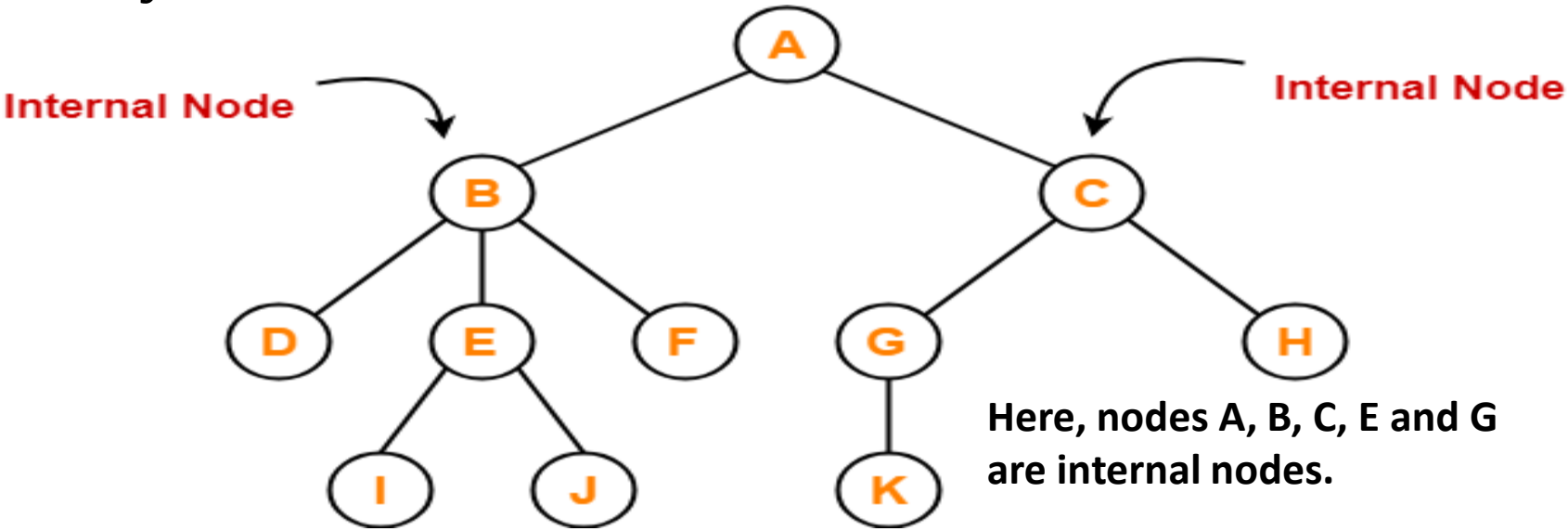
- **Degree of a node** is the total number of children of that node.
- **Degree of a tree** is the highest degree of a node among all the nodes in the tree.



Cont…

8. Internal Node

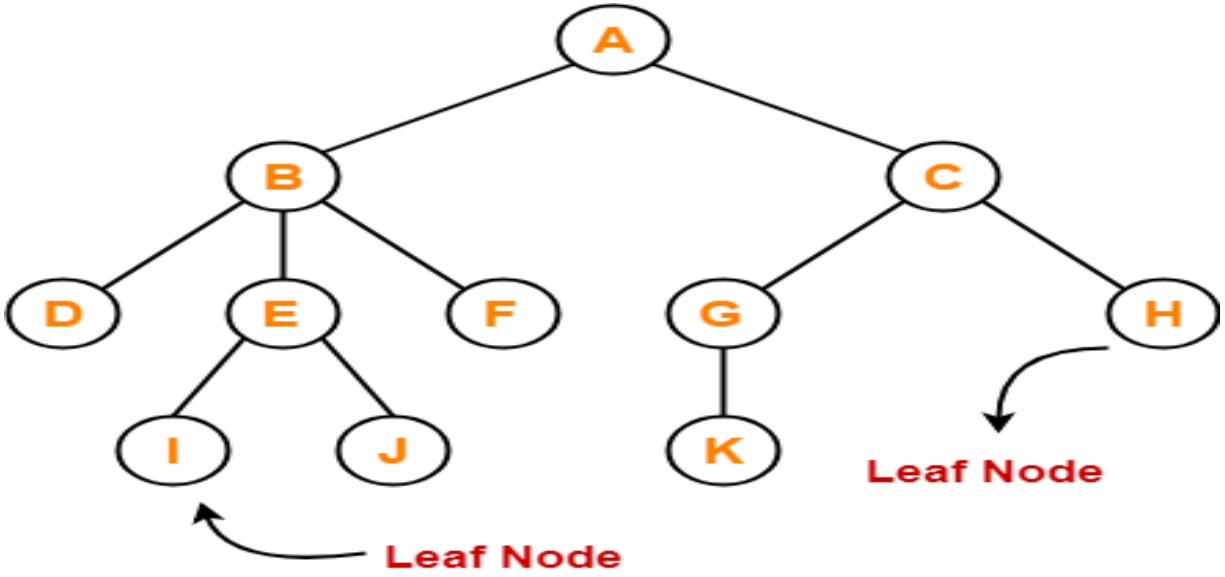
- The node which has at least one child is called as an **internal node**.
- Internal nodes are also called as **non-terminal nodes**.
- Every non-leaf node is an internal node.



Cont…

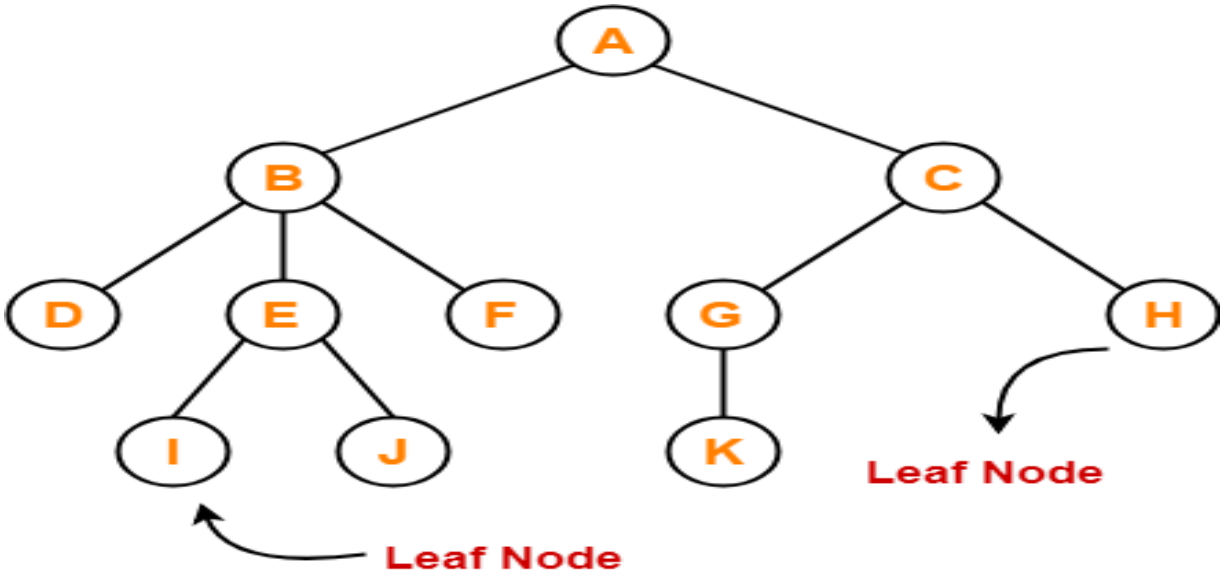
9. Leaf Node

- The node which does not have any child is called as a **leaf node**.
- Leaf nodes are also called as **external nodes** or **terminal nodes**.



10. Path

- It is a sequence of consecutive edges from the source node to the destination node .
- From given figure the **path** between A and E is given by the node pair (A,B) and (B,E).
- The no of edges in a path is called the **length of a path**.



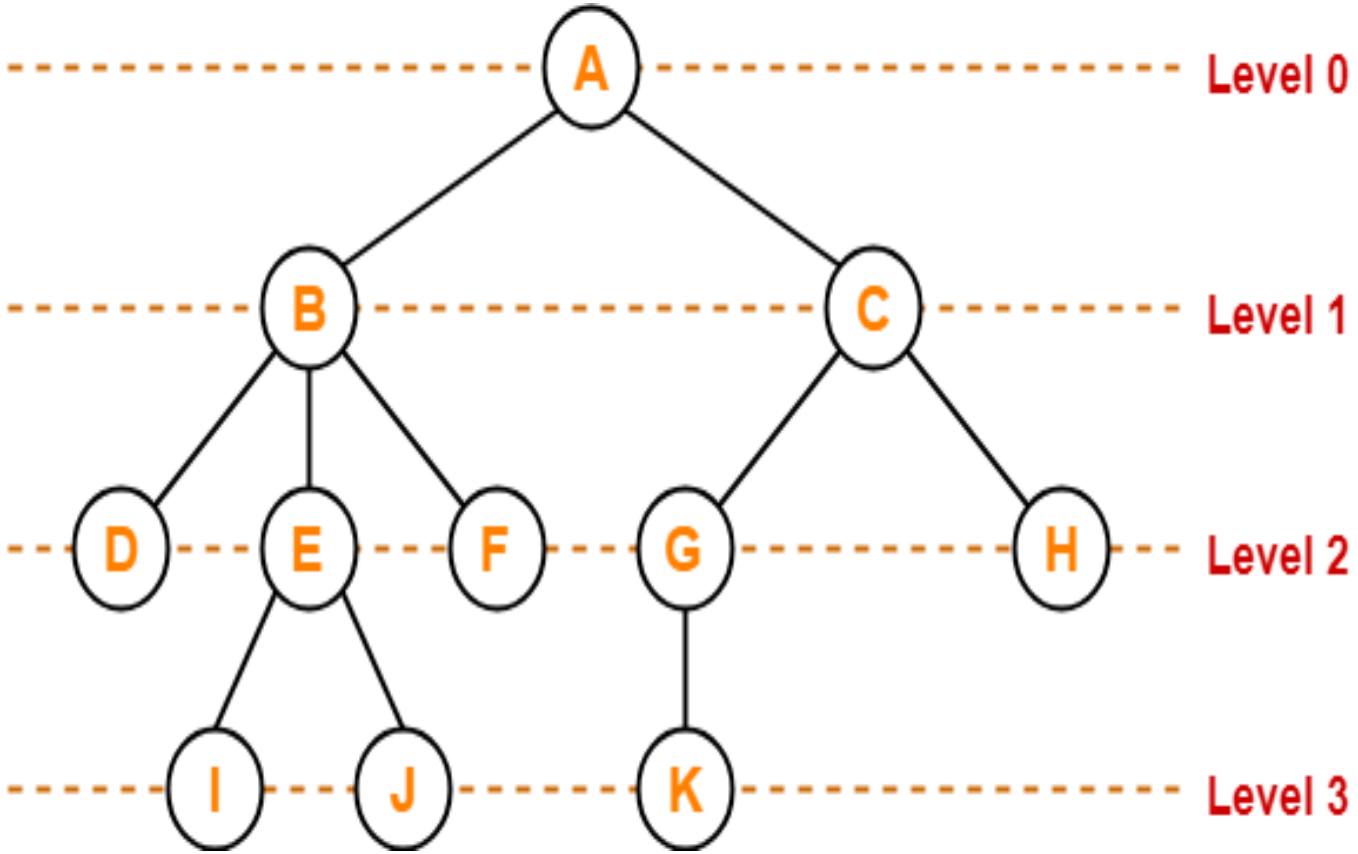
11. Level

- The level of a node is the number of edges along the unique path between that node and the root node.

- The root node is always at level 0 then its immediate children is at level 1 and their immediate children is at level 2 and so on upto the terminal node.

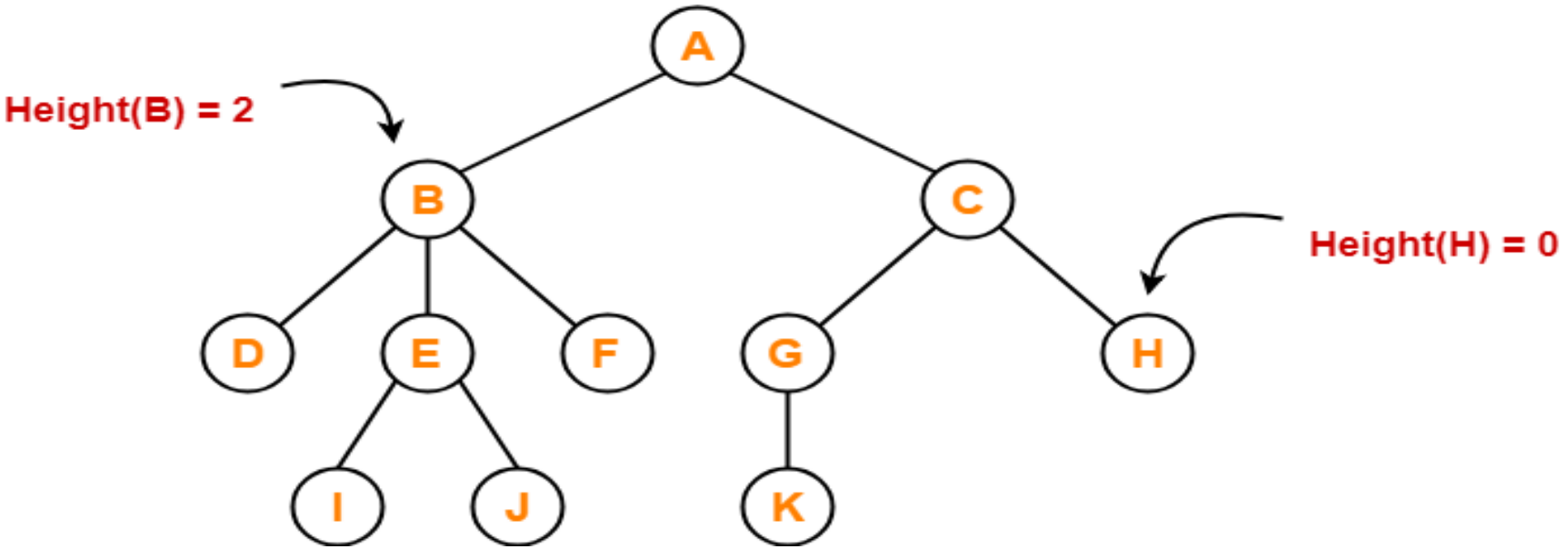
- In general if node is at level **l** then its child at level **l+1** and its parent is at level **l-1**.

- This rule is common to all nodes except root node.



12. Height

- Total number of edges that lies on the longest path from a particular node to any leaf node is called as **height of that node**.
- **Height of a tree** is the height of root node.
- Height of all leaf nodes = 0

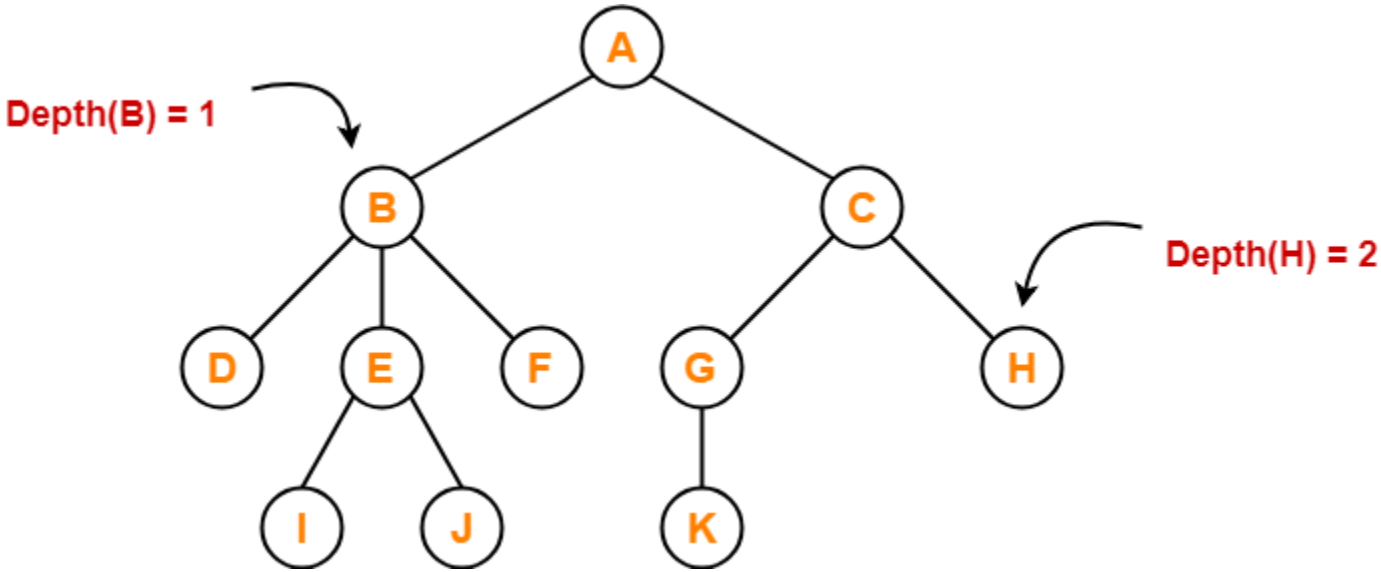


Cont…

- Height of node A = 3
- Height of node B = 2
- Height of node C = 2
- Height of node D = 0
- Height of node E = 1
- Height of node F = 0
- Height of node G = 1
- Height of node H = 0
- Height of node I = 0
- Height of node J = 0
- Height of node K = 0

13. Depth

- Total number of edges from root node to a particular node is called as **depth of that node**.
- **Depth of a tree** is the total number of edges from root node to a leaf node in the longest path.
- Depth of the root node = 0
- The terms “level” and “depth” are used interchangeably.

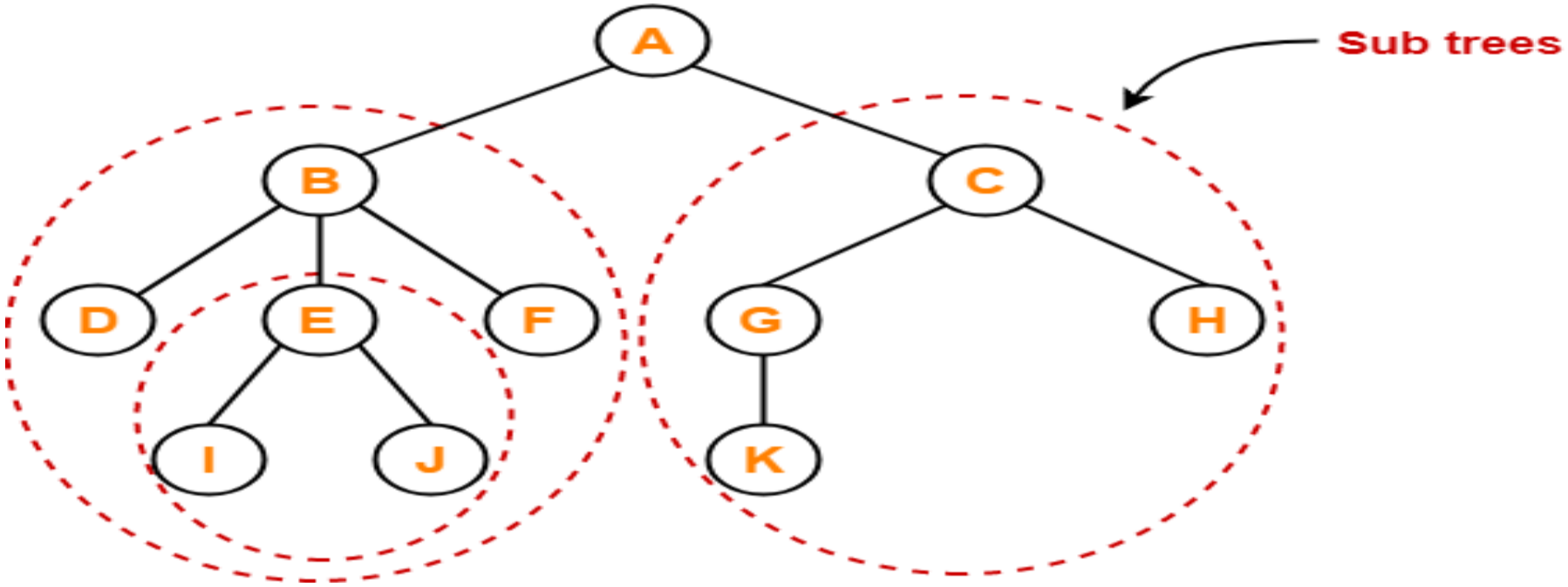


Cont…

- Depth of node A = 0
- Depth of node B = 1
- Depth of node C = 1
- Depth of node D = 2
- Depth of node E = 2
- Depth of node F = 2
- Depth of node G = 2
- Depth of node H = 2
- Depth of node I = 3
- Depth of node J = 3
- Depth of node K = 3

14. Subtree

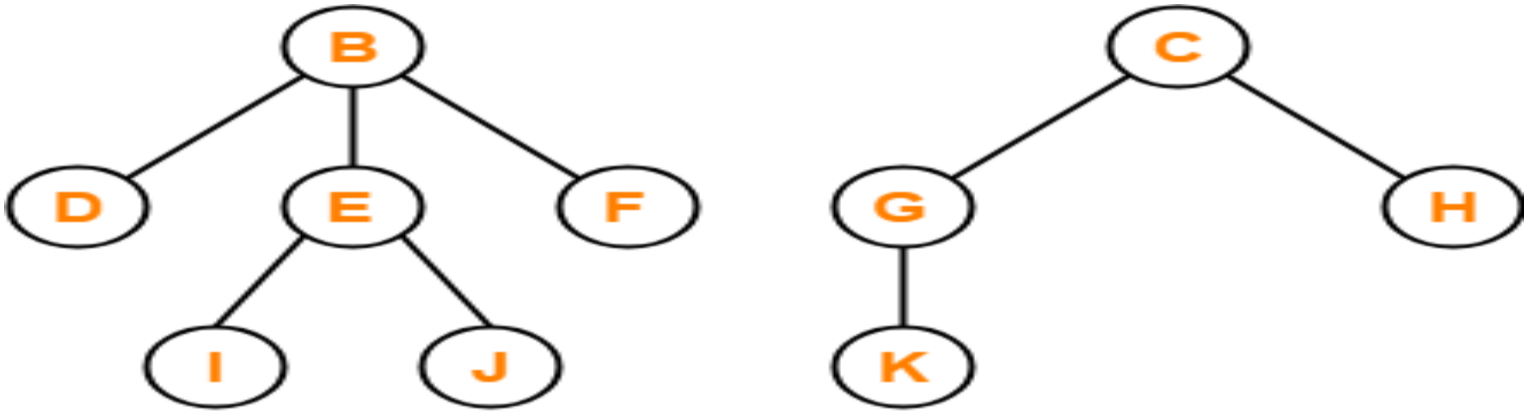
- In a tree, each child from a node forms a **subtree** recursively.



Cont…

15. Forest

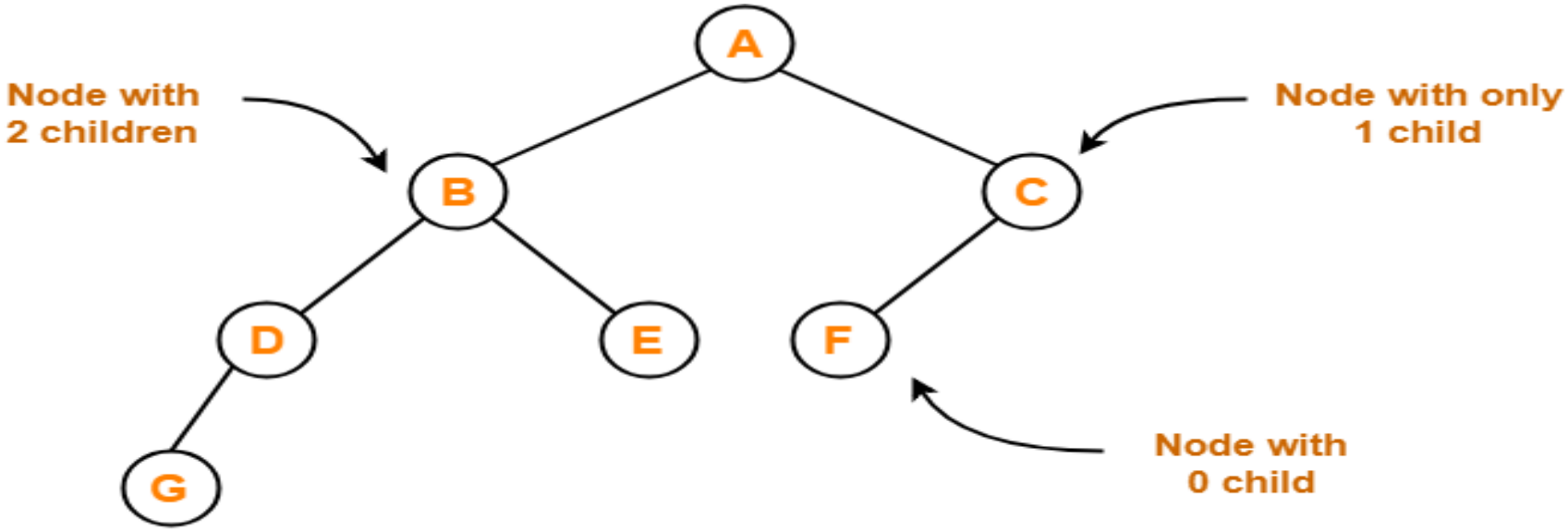
- A forest is a set of disjoint trees.



Forest

Binary Tree

- Binary tree is a special tree data structure in which each node can have at most 2 children.
- Thus, in a binary tree,
  - Each node has either 0 child or 1 child or 2 children.

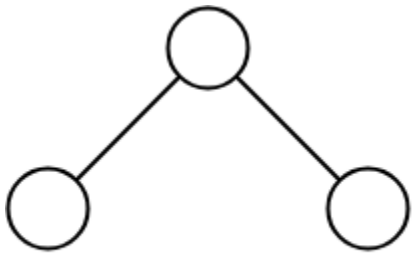


Binary Tree Example

Cont…

Unlabeled Binary Tree

- A binary tree is unlabeled if its nodes are not assigned any label.

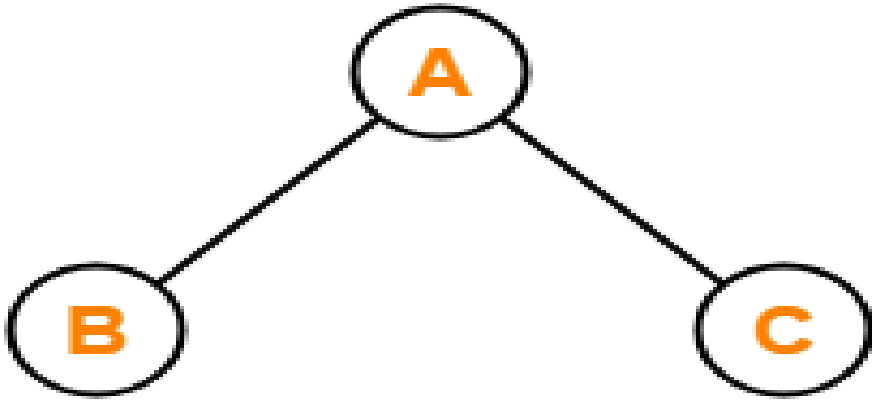


Unlabeled Binary Tree

Cont…

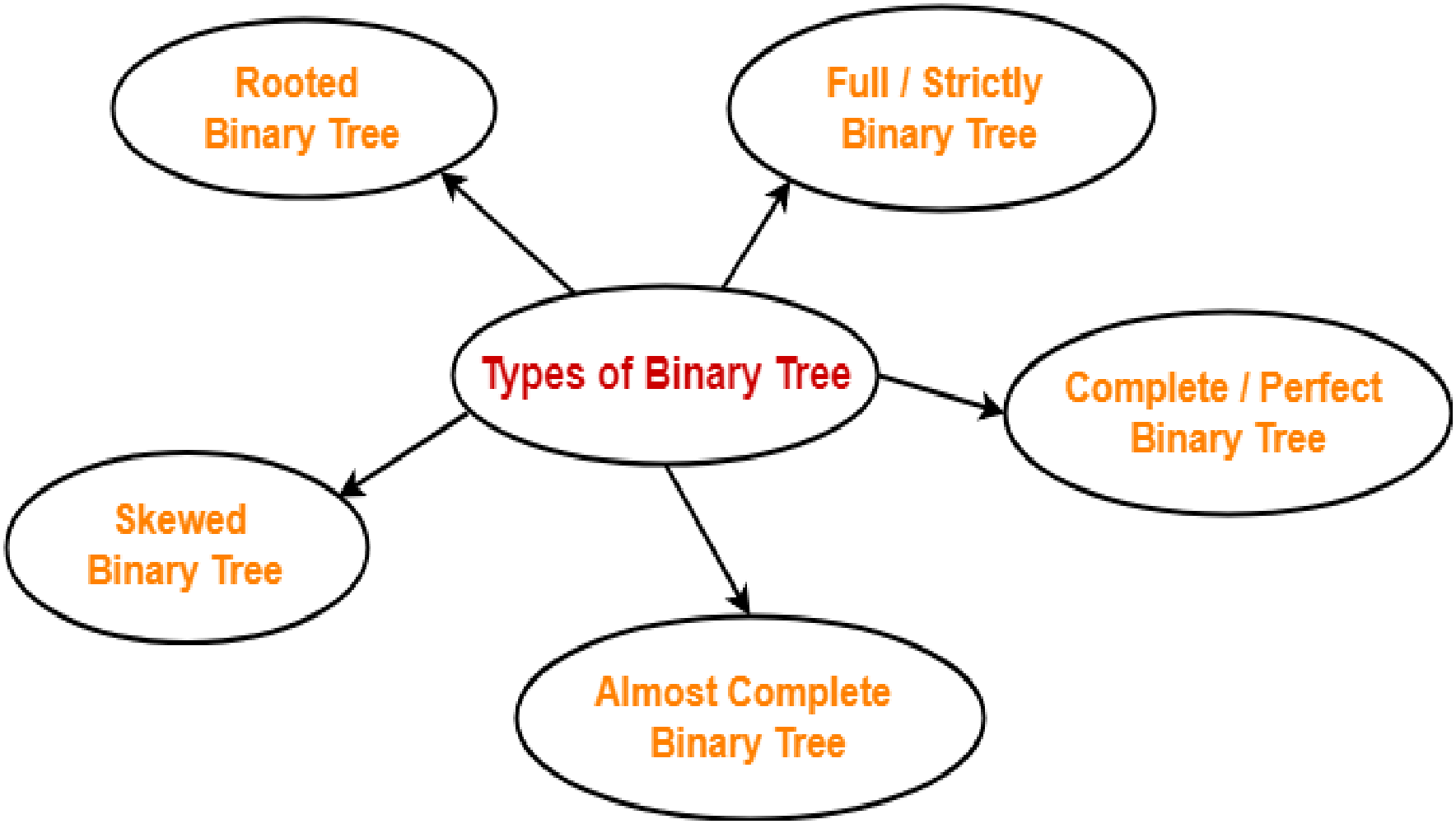
Labeled Binary Tree

- A binary tree is labeled if all its nodes are assigned a label.



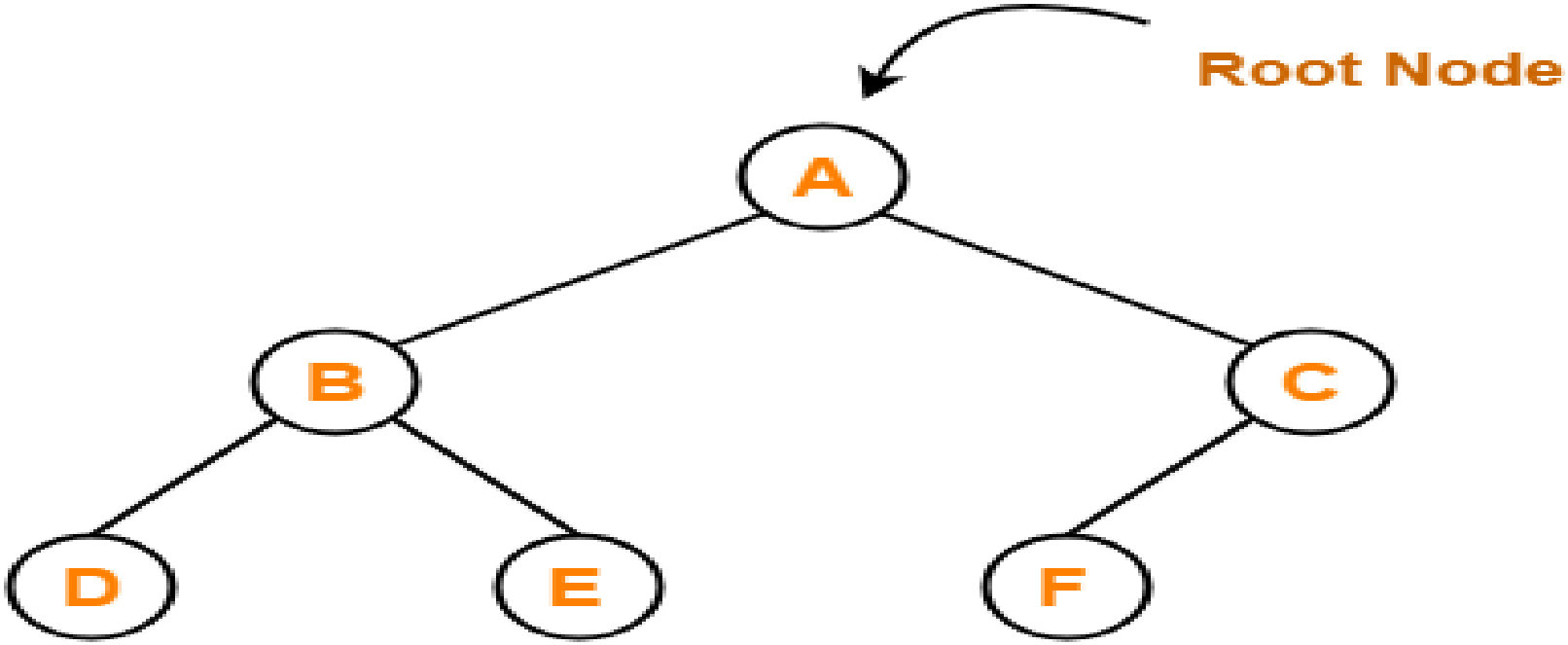
Labeled Binary Tree

Types of Binary Trees



1. Rooted Binary Tree

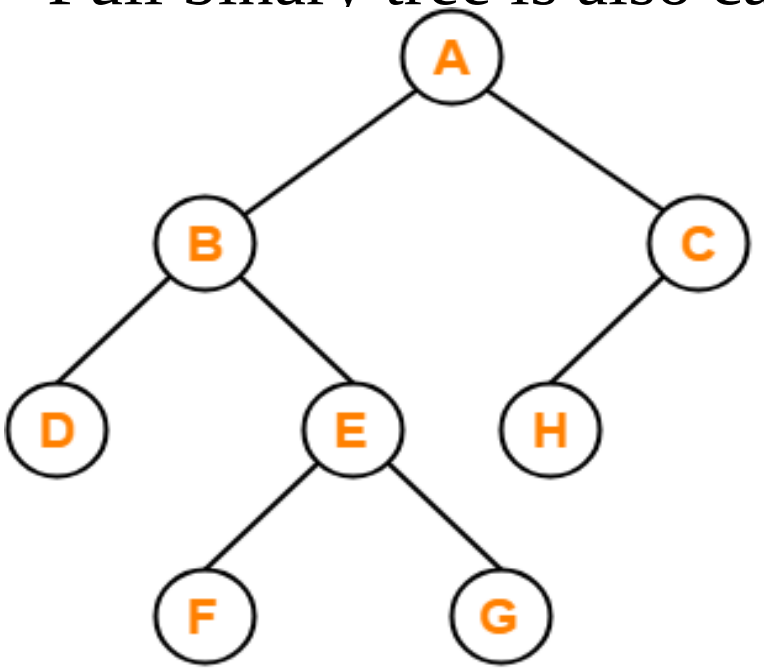
- A binary tree that satisfies the following 2 properties-
  - It has a root node.
  - Each node has at most 2 children.

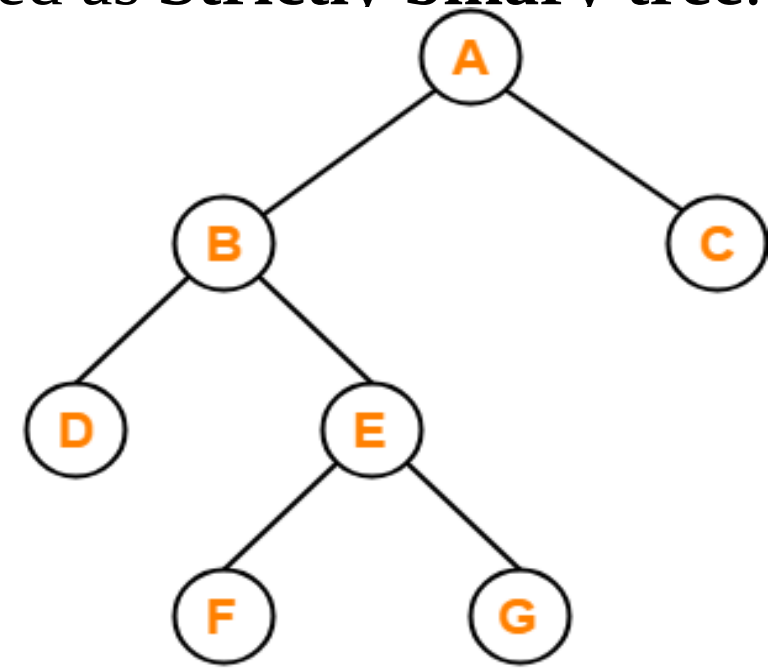


Rooted Binary Tree

2. Full / Strictly Binary Tree

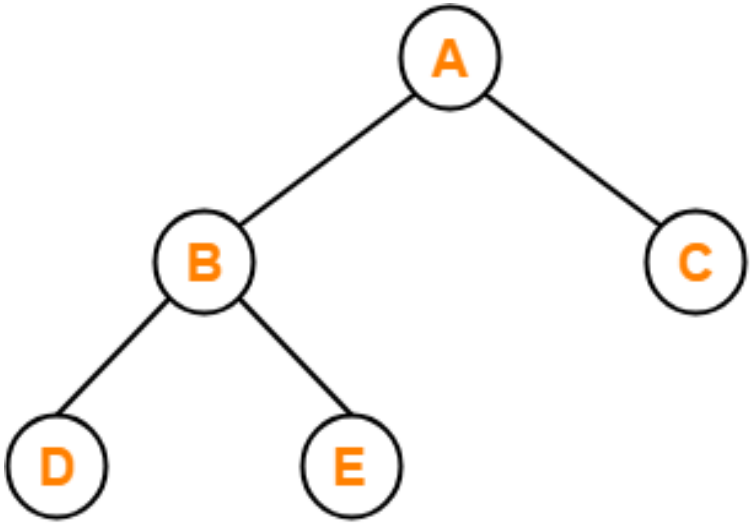
- A binary tree in which every node has either 0 or 2 children is called as a **Full binary tree**.
- Full binary tree is also called as **Strictly binary tree**.

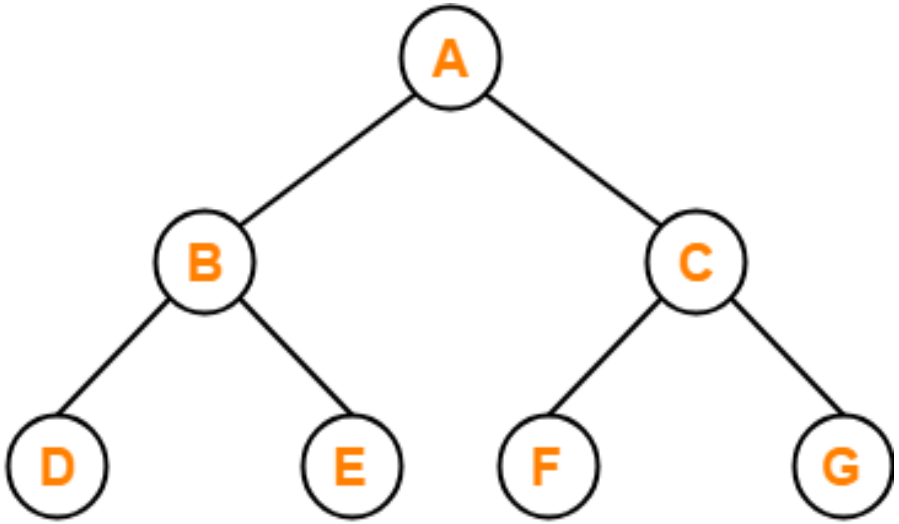




3. Complete / Perfect Binary Tree

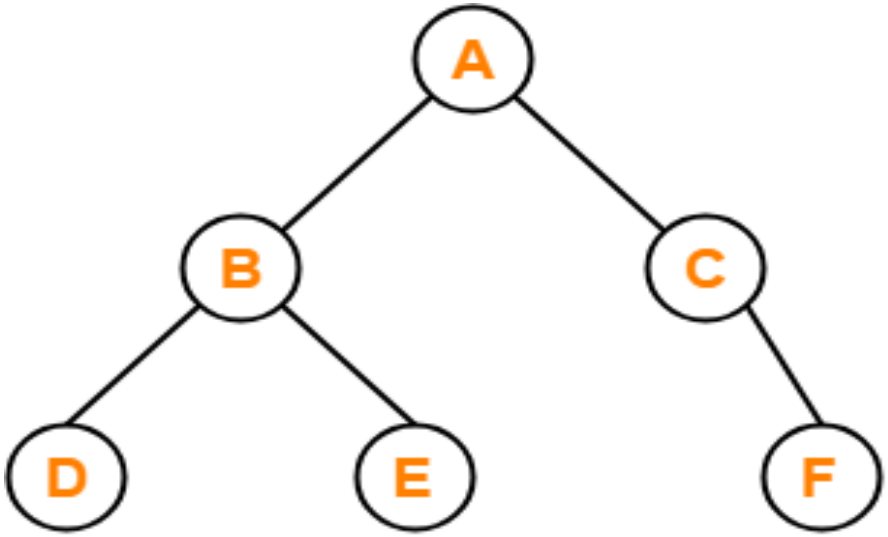
- A binary tree that satisfies the following 2 properties-
  - Every internal node has exactly 2 children.
  - All the leaf nodes are at the same level.

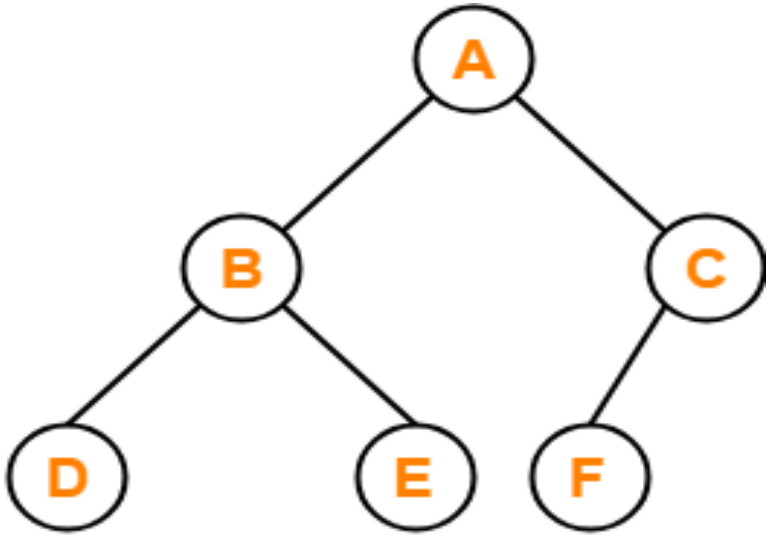




4. Almost Complete Binary Tree

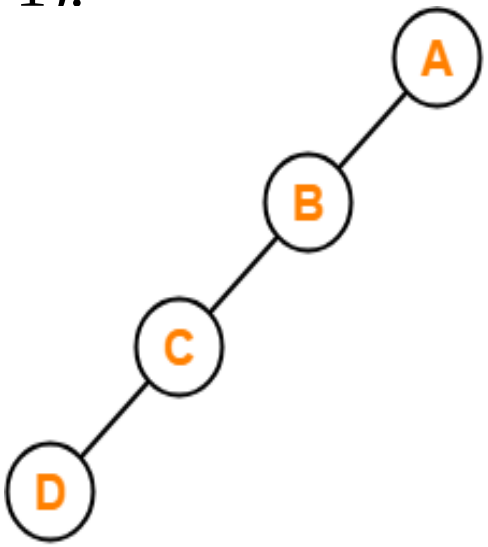
- An binary tree that satisfies the following 2 properties-
  - All the levels are completely filled except possibly the last level.
  - The last level must be strictly filled from left to right.



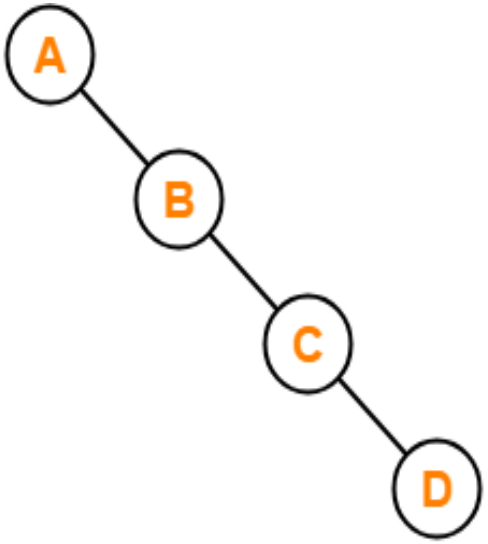


5. Skewed Binary Tree

- A binary tree that satisfies the following 2 properties-
  - All the nodes except one node has one and only one child.
  - The remaining node has no child.
  - A **skewed binary tree** is a binary tree of n nodes such that its depth is (n-1).



Left Skewed Binary Tree



Right Skewed Binary Tree

Binary Tree Properties

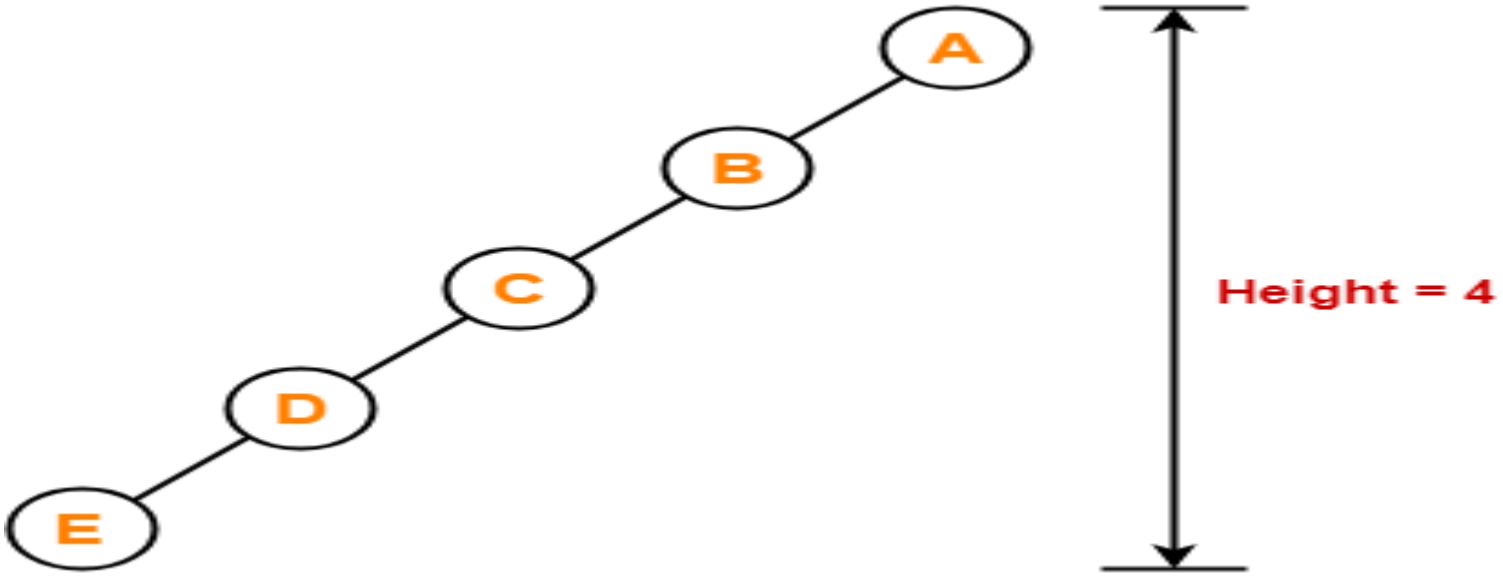
Property-01

Minimum number of nodes in a binary tree of height H = H + 1

Cont…

Example

- To construct a binary tree of height = 4, we need at least 4 + 1 = 5 nodes.



Cont…

Property-02

- **Maximum number of nodes in a binary tree of height H = $2^{H+1} - 1$**

Cont…

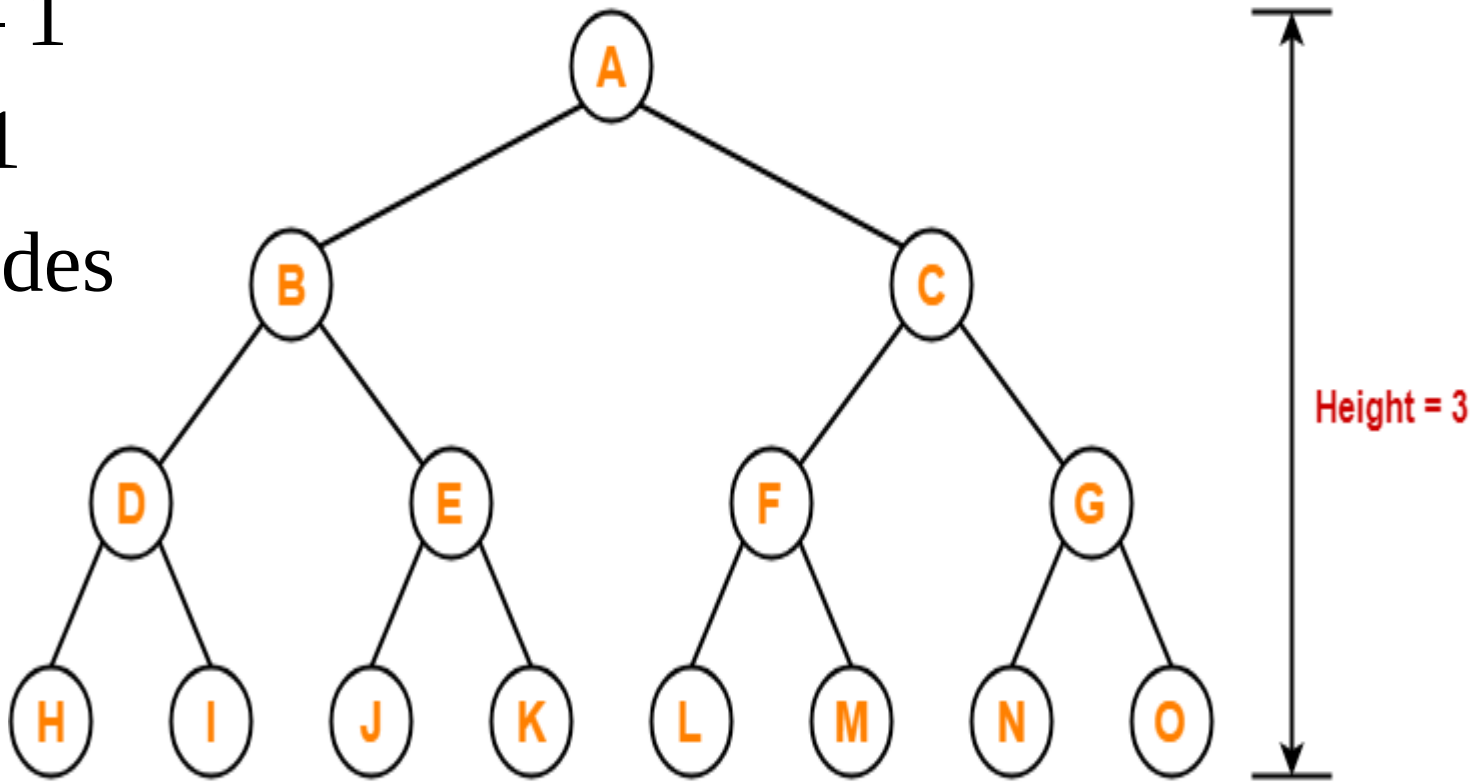
Example

- Maximum number of nodes in a binary tree of height 3

$= 2^{3+1} - 1$

$= 16 - 1$

$= 15$ nodes



Cont…

Property-03

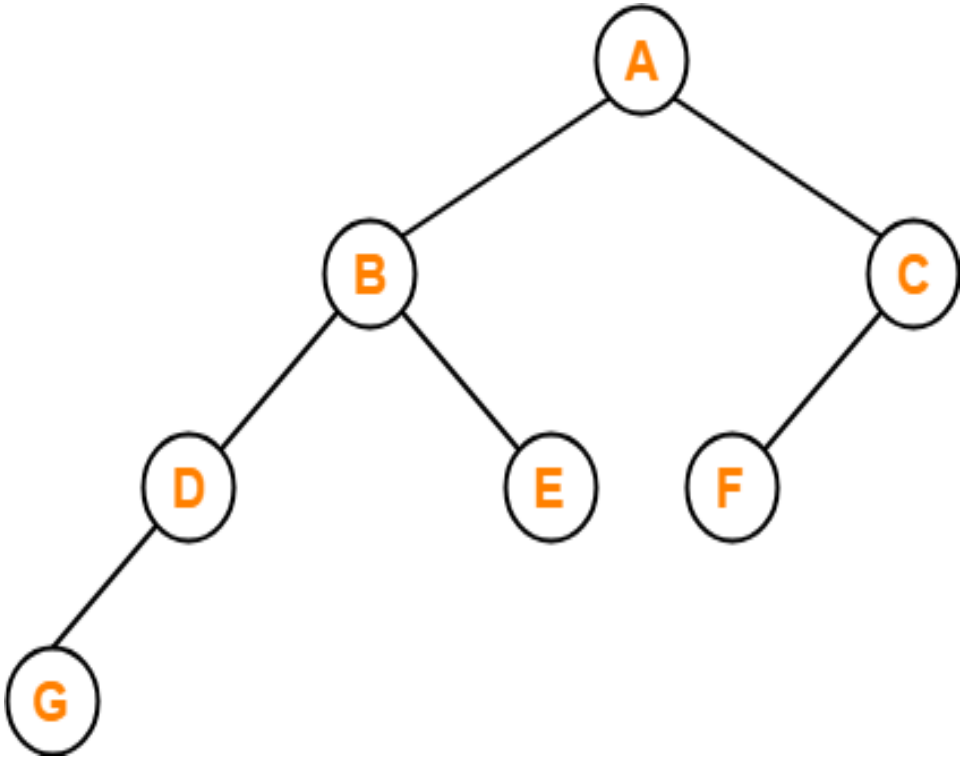
- **Total Number of leaf nodes in a Binary Tree = Total Number of nodes with 2 children + 1**

Cont…

Example

Here,

- Number of nodes with 2 children = 2
- Number of leaf nodes = Number of nodes with 2 children + 1= 2+1=3
- Number of leaf nodes = 3



Cont…

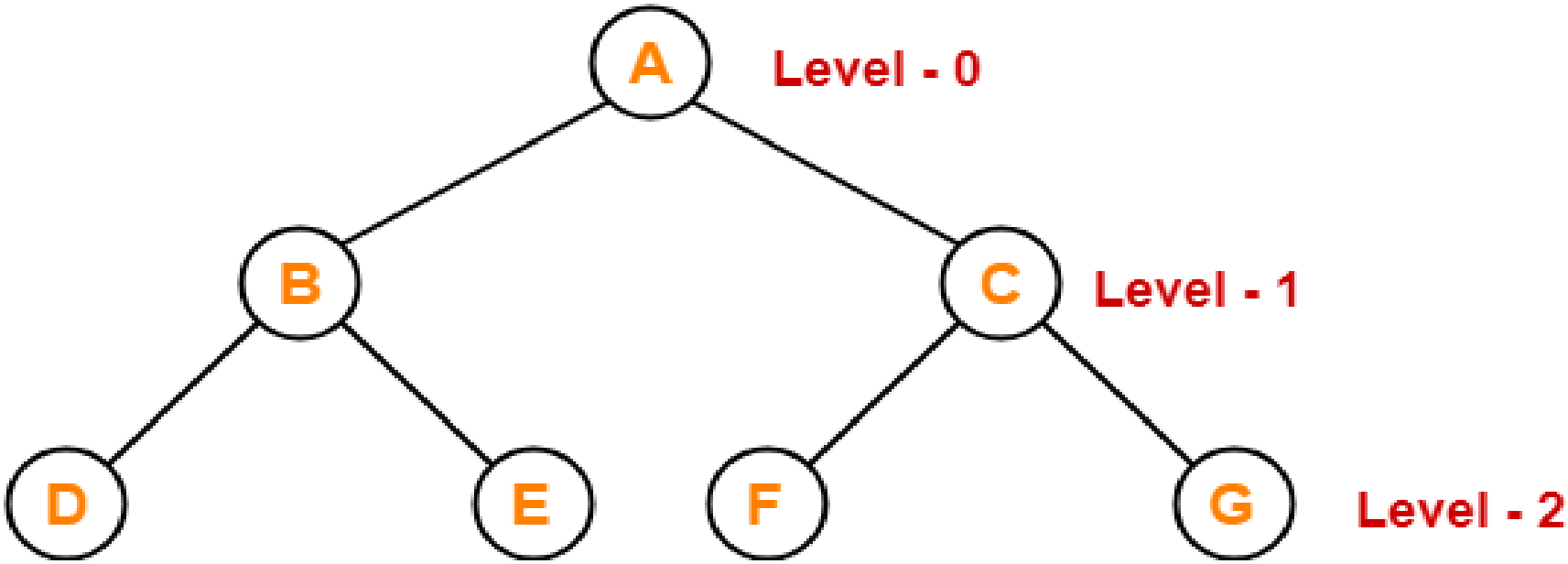
Property-04

- **Maximum number of nodes at any level ‘L’ in a binary tree = 2^L**

Cont…

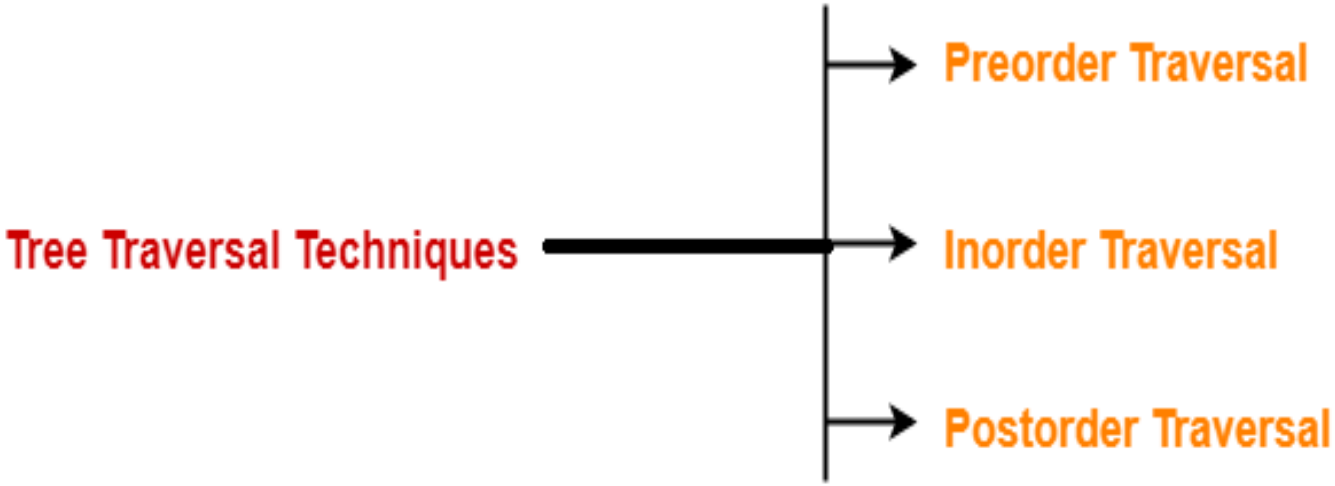
Example

- Maximum number of nodes at level-2 in a binary tree = $2^2 = 4$
- Thus, in a binary tree, maximum number of nodes that can be present at level-2 = 4.



Tree Traversal

- It is a way in which each node in the tree is visited exactly once in a systematic manner.



1. Preorder Traversal (NLR)

Algorithm

- Visit the node.
- Traverse the left sub tree in preorder.
- Traverse the right sub tree in preorder.

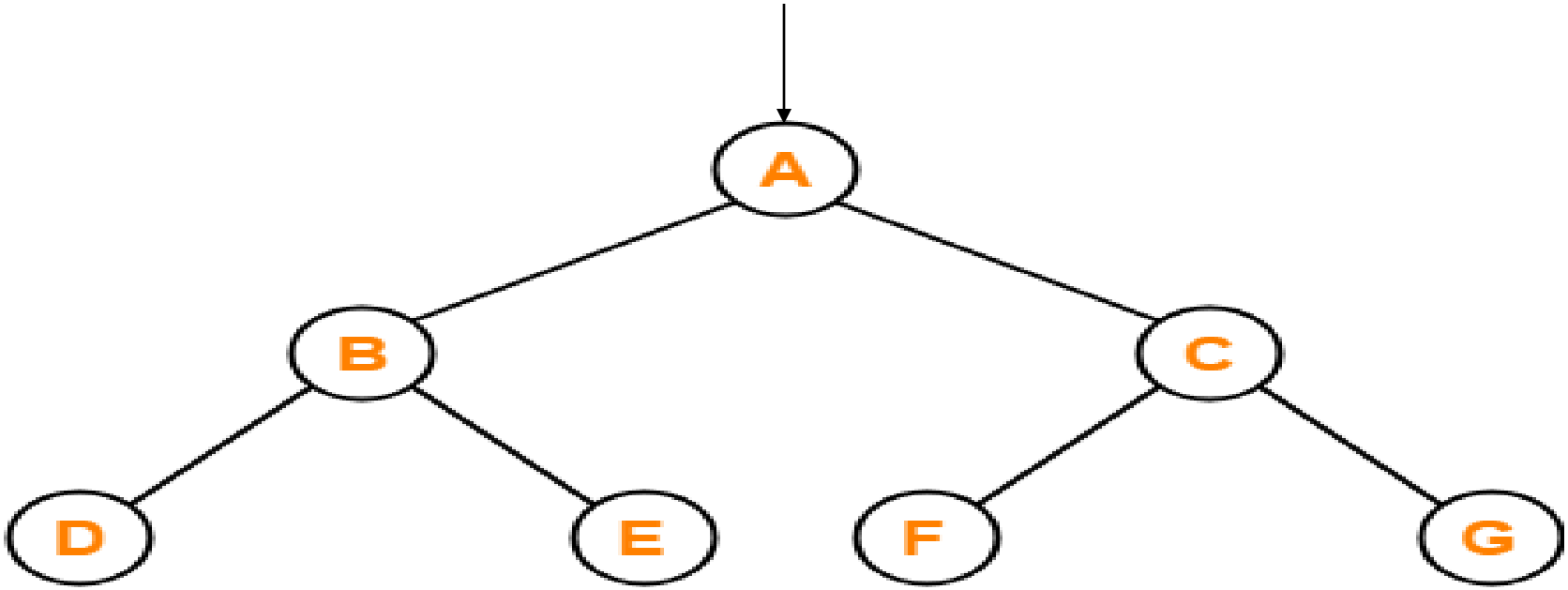
Node → Left → Right

✅ 1. Preorder Traversal (Root → Left → Right)

Algorithm PreorderTraversal(root)
Input: root node of the binary tree
Output: list of node values in preorder traversal

1. Initialize empty stack S
2. Initialize empty list result
3. If root is null, return result
4. Push root onto stack S
5. While S is not empty:
    a. Pop node from S and add node.data to result
    b. If node.right is not null, push node.right to S
    c. If node.left is not null, push node.left to S
6. Return result

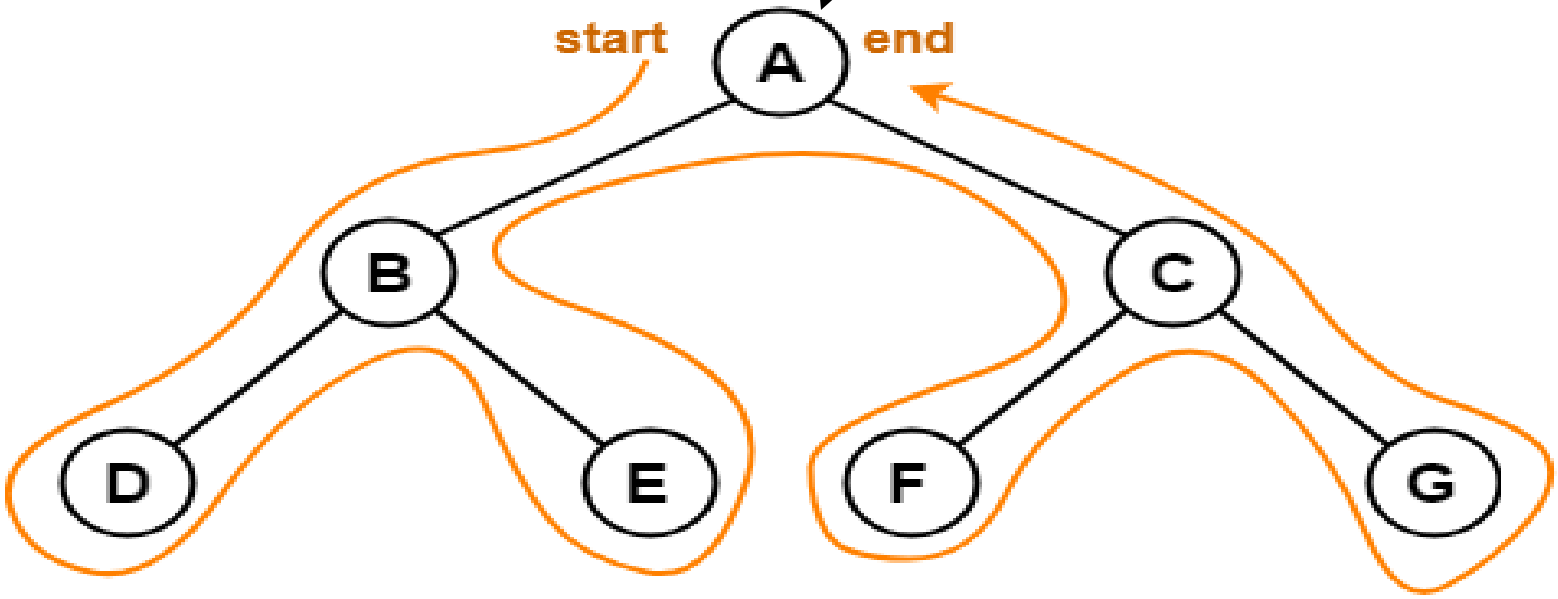
Cont…



Preorder Traversal : A , B , D , E , C , F , G

Cont…

- Traverse the entire tree starting from the root node keeping yourself to the left.



Preorder Traversal : A , B , D , E , C , F , G

Cont…

Applications

- Preorder traversal is used to get prefix expression of an expression tree.

- Preorder traversal is used to create a copy of the tree.

Cont…

Note: **First node** in a preorder traversal is a **root node** in a Binary Tree.

2. Inorder Traversal (LNR)

Algorithm

- Traverse the left sub tree in inorder.
- Visit the node.
- Traverse the right sub tree in inorder.

Left → Node → Right

2. Inorder Traversal (LNR)

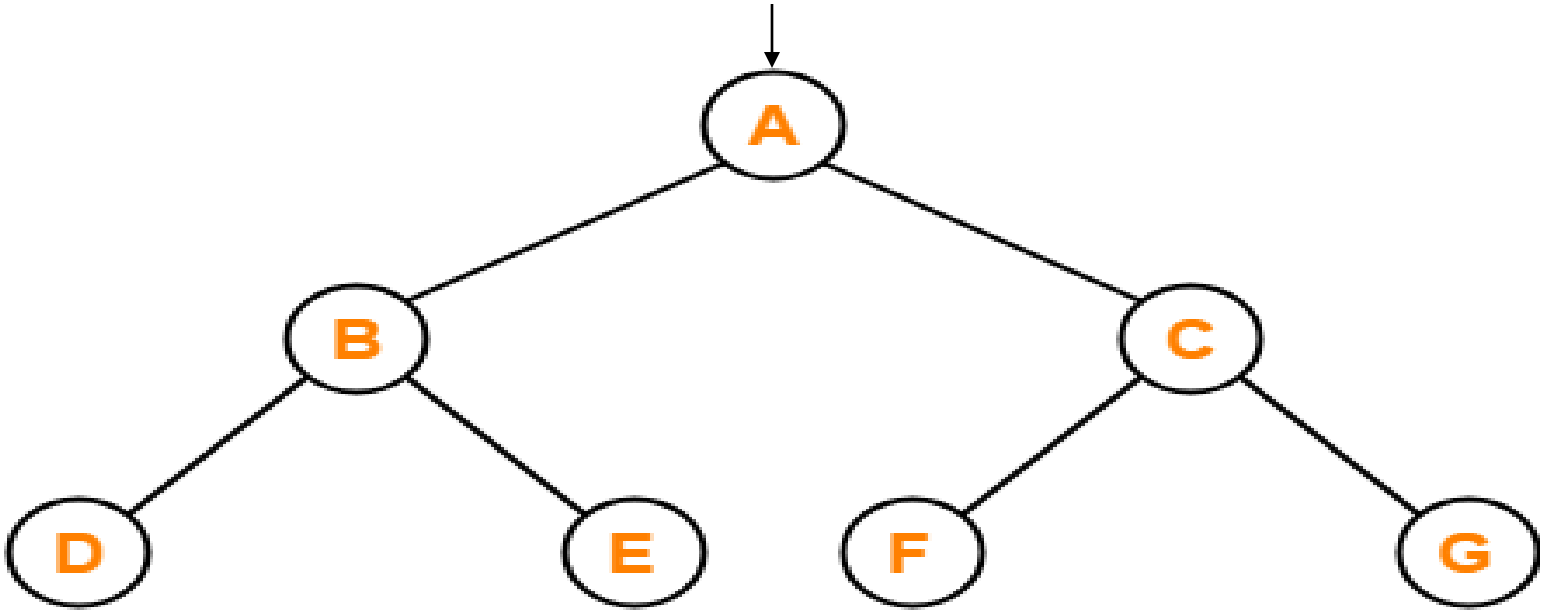
Algorithm InorderTraversal(root)

Input: root node of the binary tree

Output: list of node values in inorder traversal

- 1. Initialize empty stack S
- 2. Initialize empty list result
- 3. Set current = root
- 4. While current is not null OR S is not empty:
- a. While current is not null:
- i. Push current to S
- ii. current = current.left
- b. Pop from S and set current = popped node
- c. Add current.data to result
- d. current = current.right
- 5. Return result

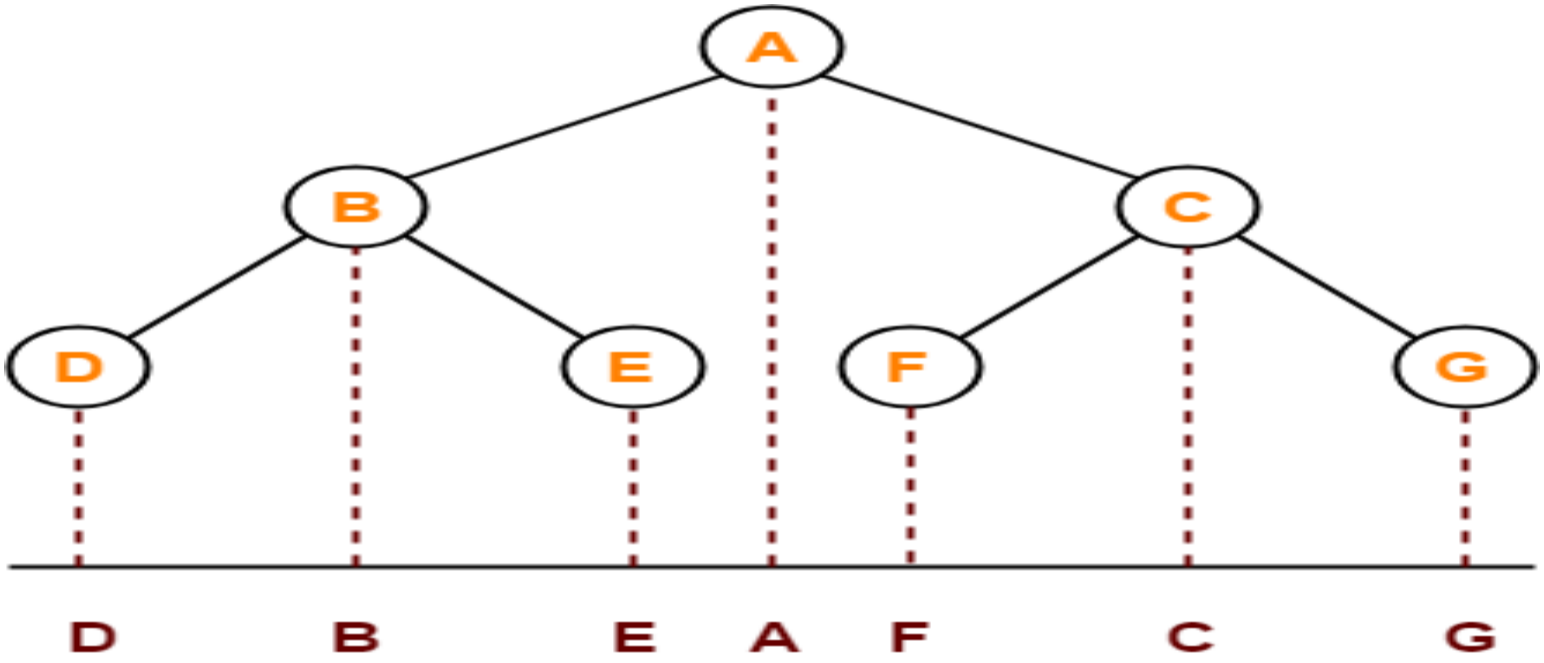
Cont…



Inorder Traversal : D , B , E , A , F , C , G

Cont…

- Keep a plane mirror horizontally at the bottom of the tree and take the projection of all the nodes.



Inorder Traversal : D , B , E , A , F , C , G

Cont…

Application

- Inorder traversal is used to get infix expression of an expression tree.

3. Postorder Traversal (LRN)

Algorithm

- Traverse the left sub tree in postorder.
- Traverse the right sub tree in postorder.
- Visit the node.

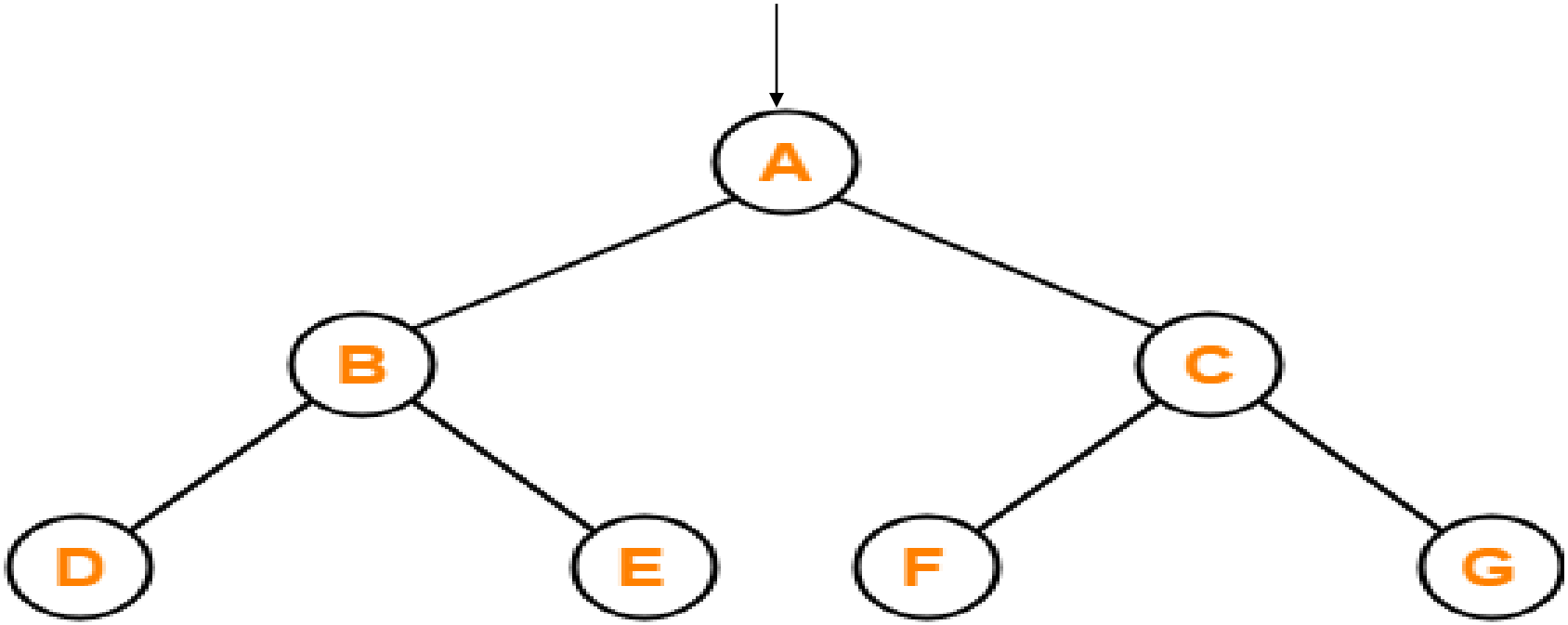
Left → Right → Node

3. Postorder Traversal (LRN)

Algorithm PostorderTraversal(root)
Input: root node of the binary tree
Output: list of node values in postorder traversal

1. Initialize two stacks: S1, S2
2. Initialize empty list result
3. If root is null, return result
4. Push root to S1
5. While S1 is not empty:
    a. Pop node from S1 and push it to S2
    b. If node.left is not null, push node.left to S1
    c. If node.right is not null, push node.right to S1
6. While S2 is not empty:
    a. Pop node from S2 and add node.data to result
7. Return result

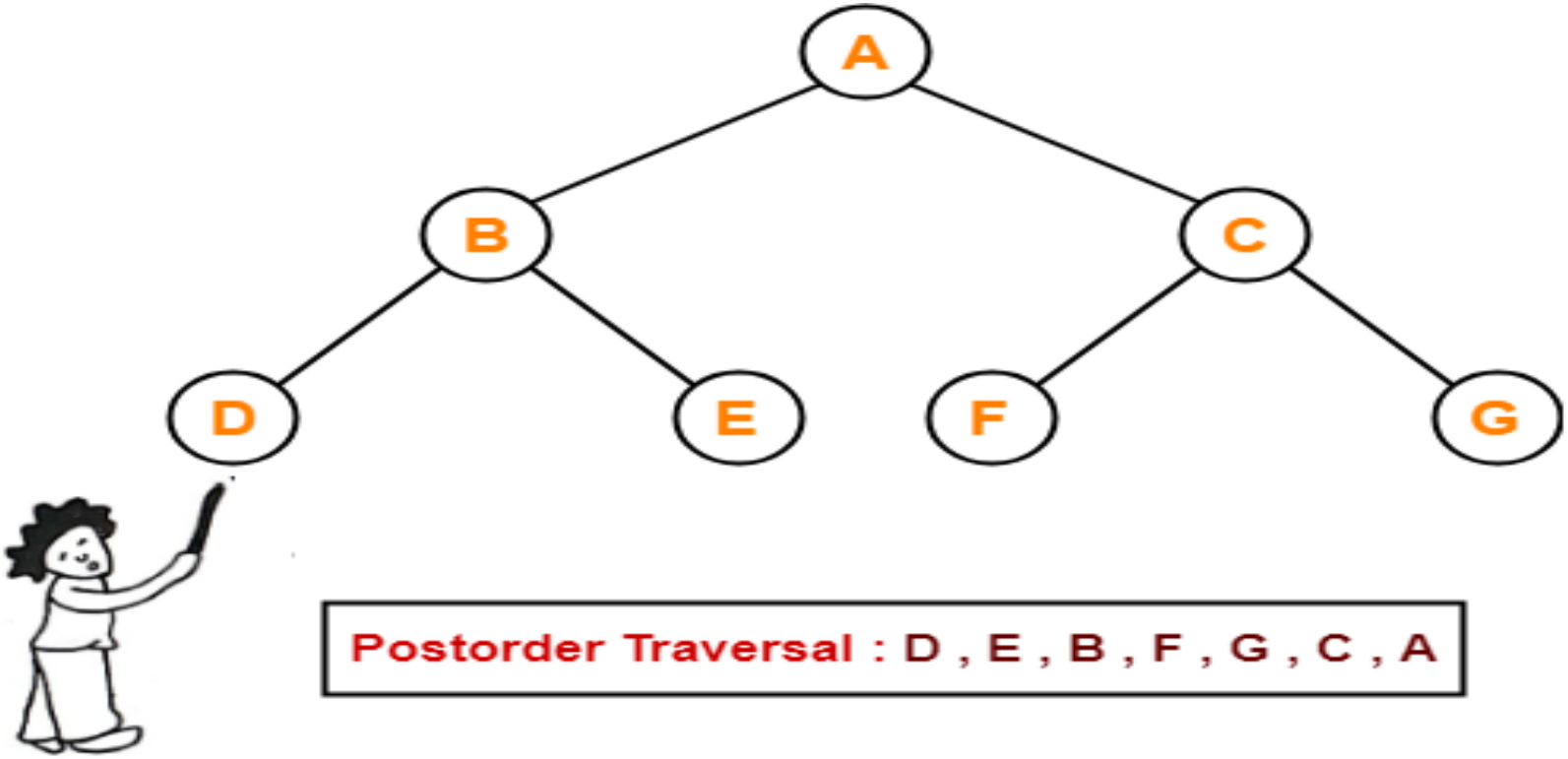
Cont…



Postorder Traversal : D , E , B , F , G , C , A

Cont…

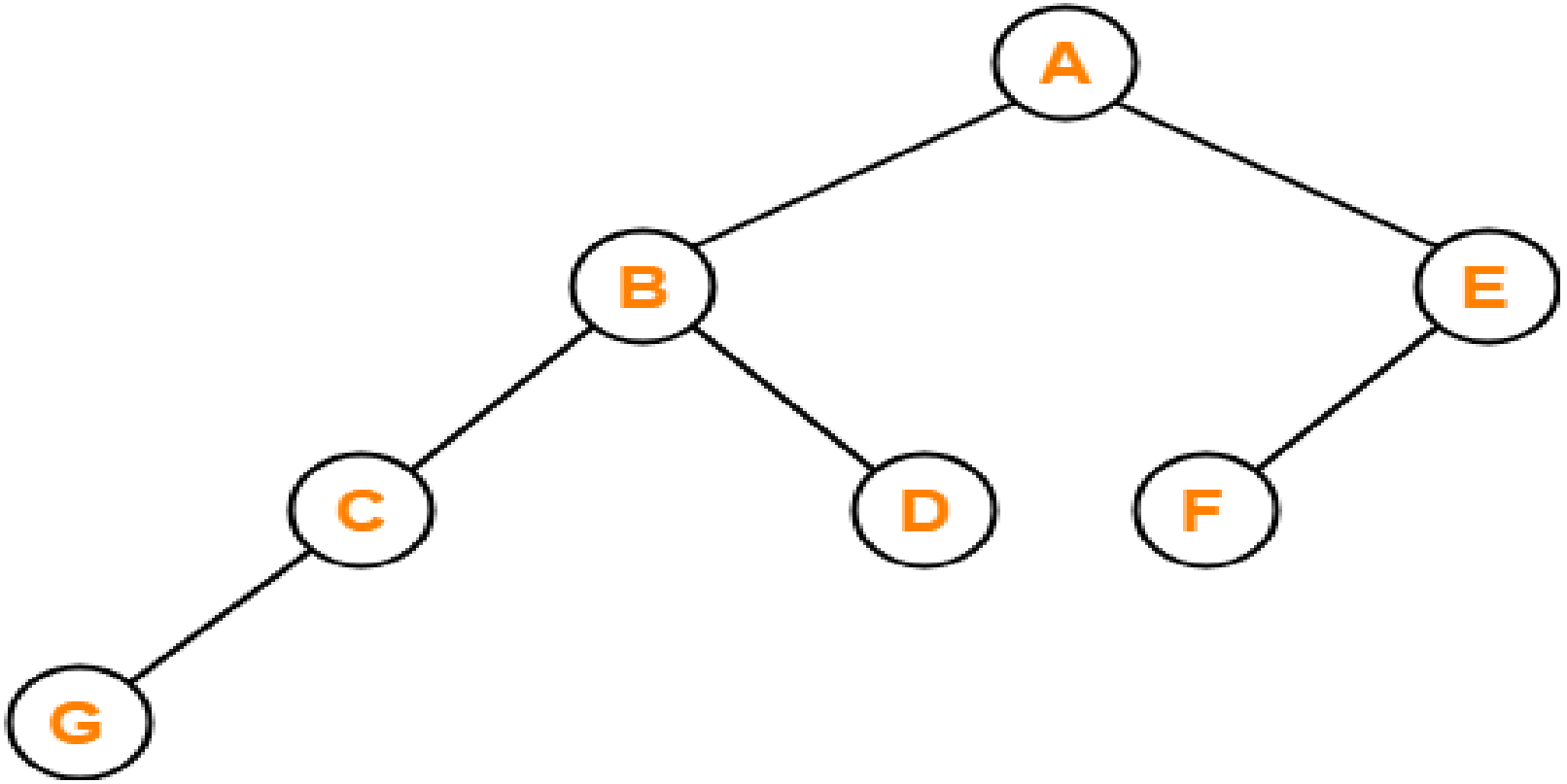
- Pluck all the leftmost leaf nodes one by one.



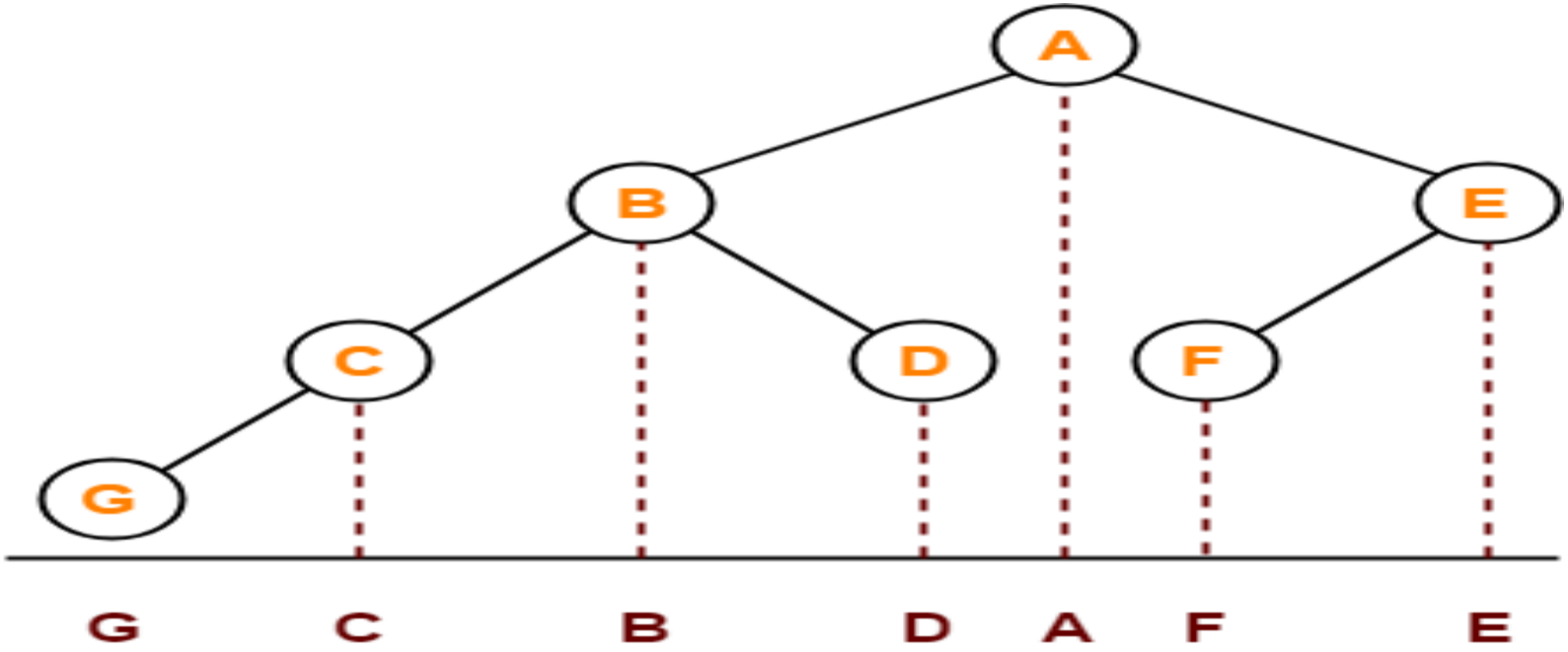
Cont…

Note: **Last node** in a postorder traversal is a **root node** in a Binary Tree.

Problem-01: Find Inorder of the following binary tree



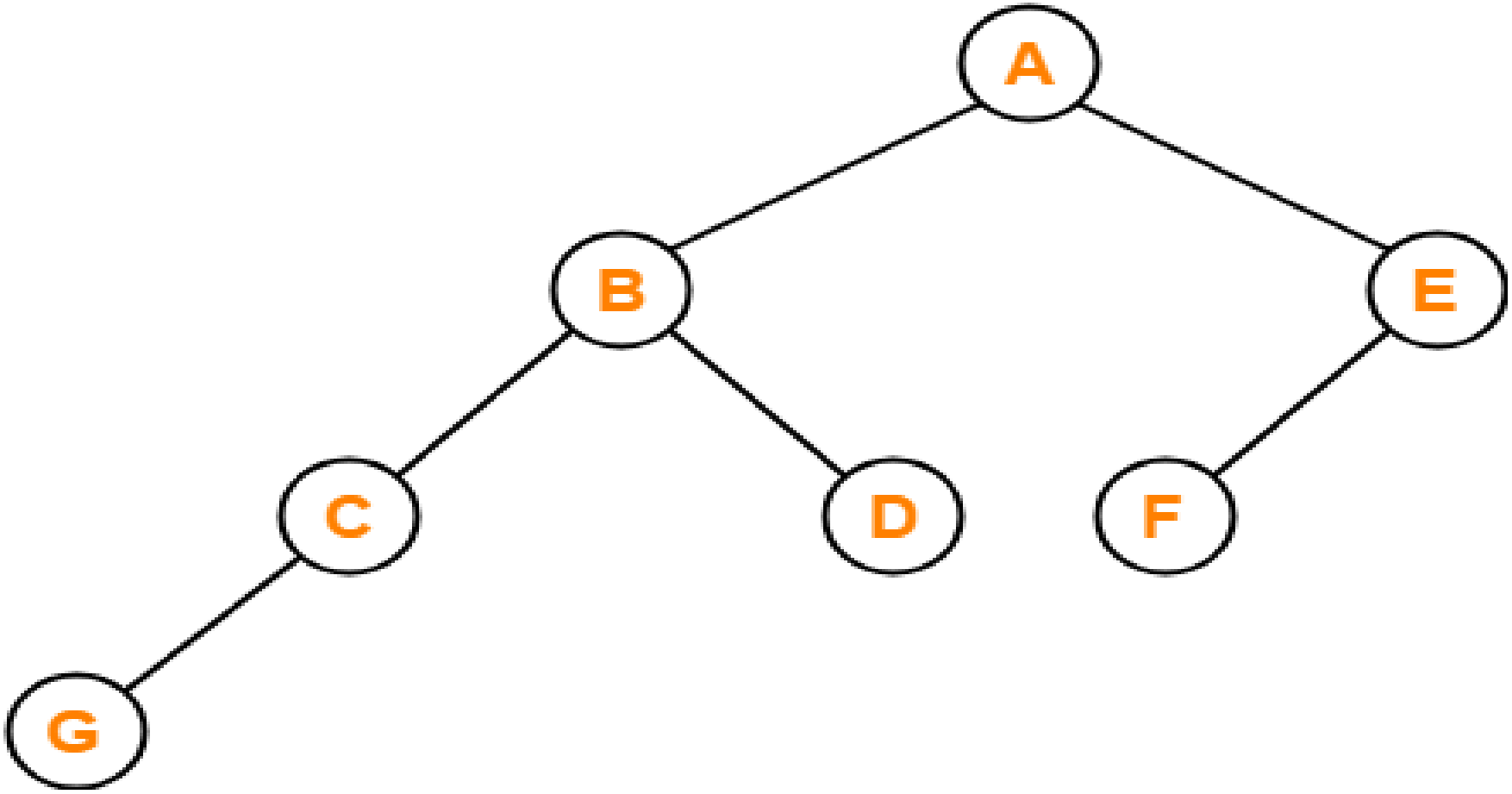
Solution



Inorder Traversal : G , C , B , D , A , F , E

Problem-02:

Find the postorder and preorder traversal sequence of the given binary tree



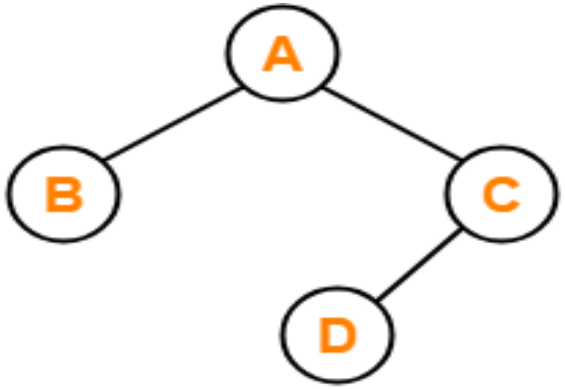
Solution

- Postorder Traversal : G , C , D , B , F , E , A

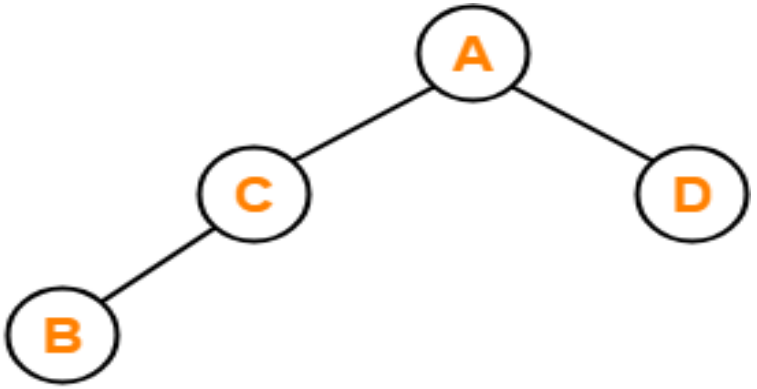
- Preorder Traversal: A, B, C, G, D, E, F

Problem-04: Which of the following binary trees has its inorder and preorder traversals as BCAD and ABCD respectively-

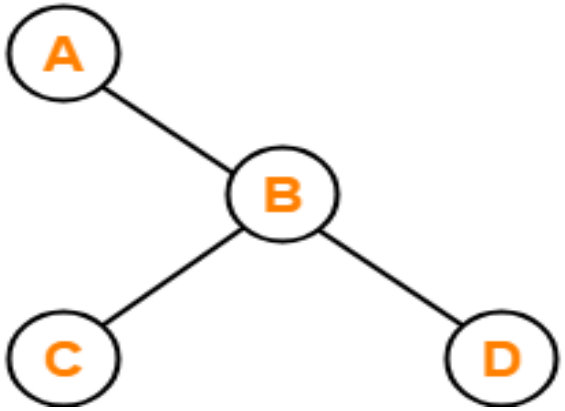
A)



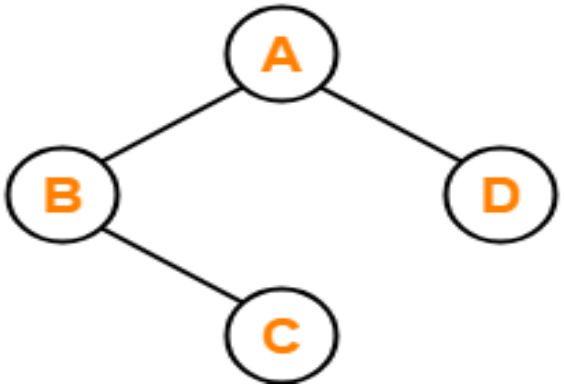
B)



C)



D)



Solution

- Option (D) is correct.

Tree Traversal Algorithms

- Binary Tree Traversals Overview
- 🔍 Definitions:
- • Preorder: Visit Root → Left → Right
- • Inorder: Visit Left → Root → Right
- • Postorder: Visit Left → Right → Root
- _____
- ✅ Input and Output
- • Input: A binary tree (either as a reference to its root node or structured nodes).
- • Output: A list or sequence of node values in the respective traversal order.
- _____

Tree Traversal Algorithms

- **1. Preorder Traversal**
- Recursion Algorithm:
- Algorithm (Preorder_Recursive)
- Input: Root node of binary tree
- Output: Nodes in preorder (Root, Left, Right)

- 1. If root is NULL, return
- 2. Visit(root)
- 3. Preorder_Recursive(root.left)
- 4. Preorder_Recursive(root.right)

Tree Traversal Algorithms

- **2. Inorder Traversal**
- Recursion Algorithm:
- Algorithm (Inorder_Recursive)
- Input: Root node of binary tree
- Output: Nodes in inorder (Left, Root, Right)

- 1. If root is NULL, return
- 2. Inorder_Recursive(root.left)
- 3. Visit(root)
- 4. Inorder_Recursive(root.right)

Tree Traversal Algorithms

- **3. Postorder Traversal**
- Recursion Algorithm:
- Algorithm (Postorder_Recursive)
- Input: Root node of binary tree
- Output: Nodes in postorder (Left, Right, Root)

- 1. If root is NULL, return
- 2. Postorder_Recursive(root.left)
- 3. Postorder_Recursive(root.right)
- 4. Visit(root)

Creating Binary Tree by using Traversal

- There are two different way for creating binary tree.

(1) Preorder and Inorder Traversal.

(2) Postorder and Inorder Traversal.

(1) Preorder and Inorder Traversal

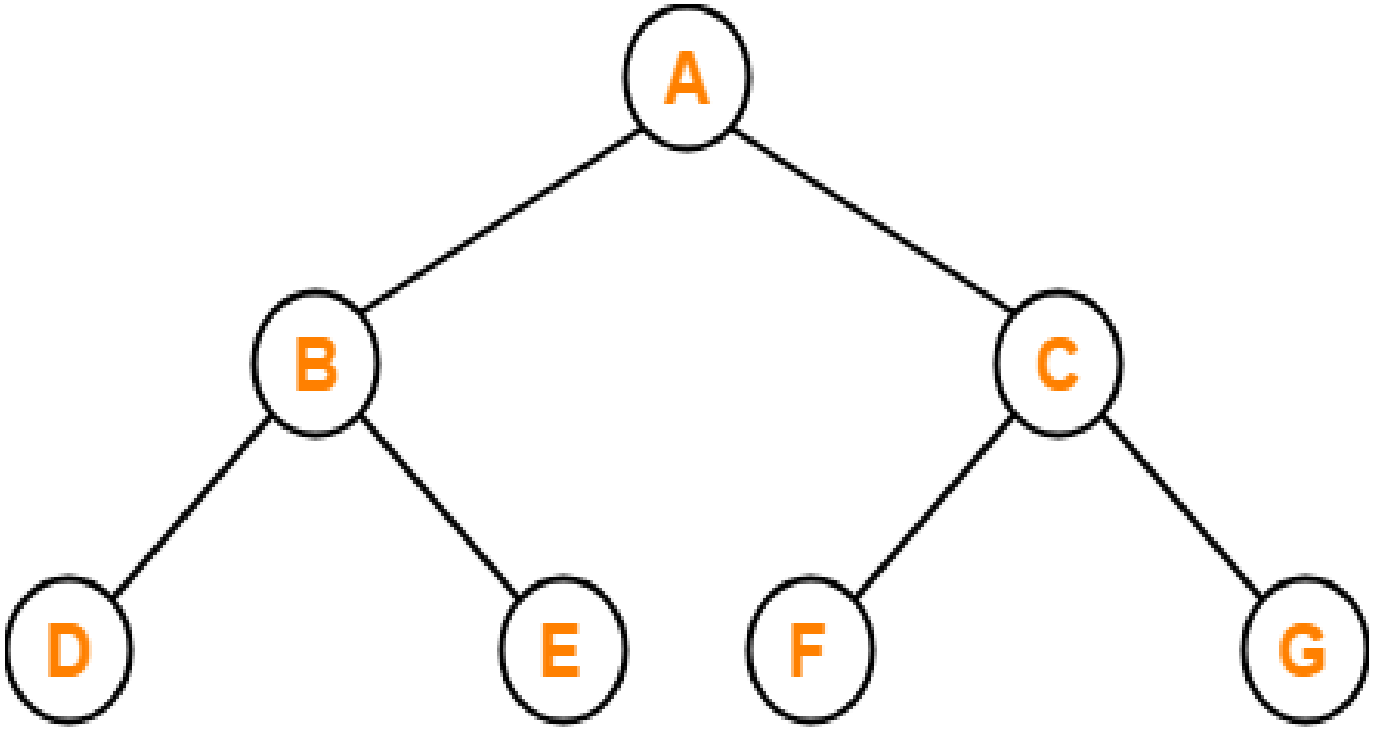
Q: Construct the binary tree given the following traversal.

Preorder: **A, B, D, E, C, F, G**

Inorder: **D, B, E, A, F, C, G**

Preorder: **A, B, D, E, C, F, G**
Inorder: **D, B, E, A, F, C, G**

Ans:



(2) Postorder and Inorder Traversal

Q: Construct the binary tree given the following traversal.

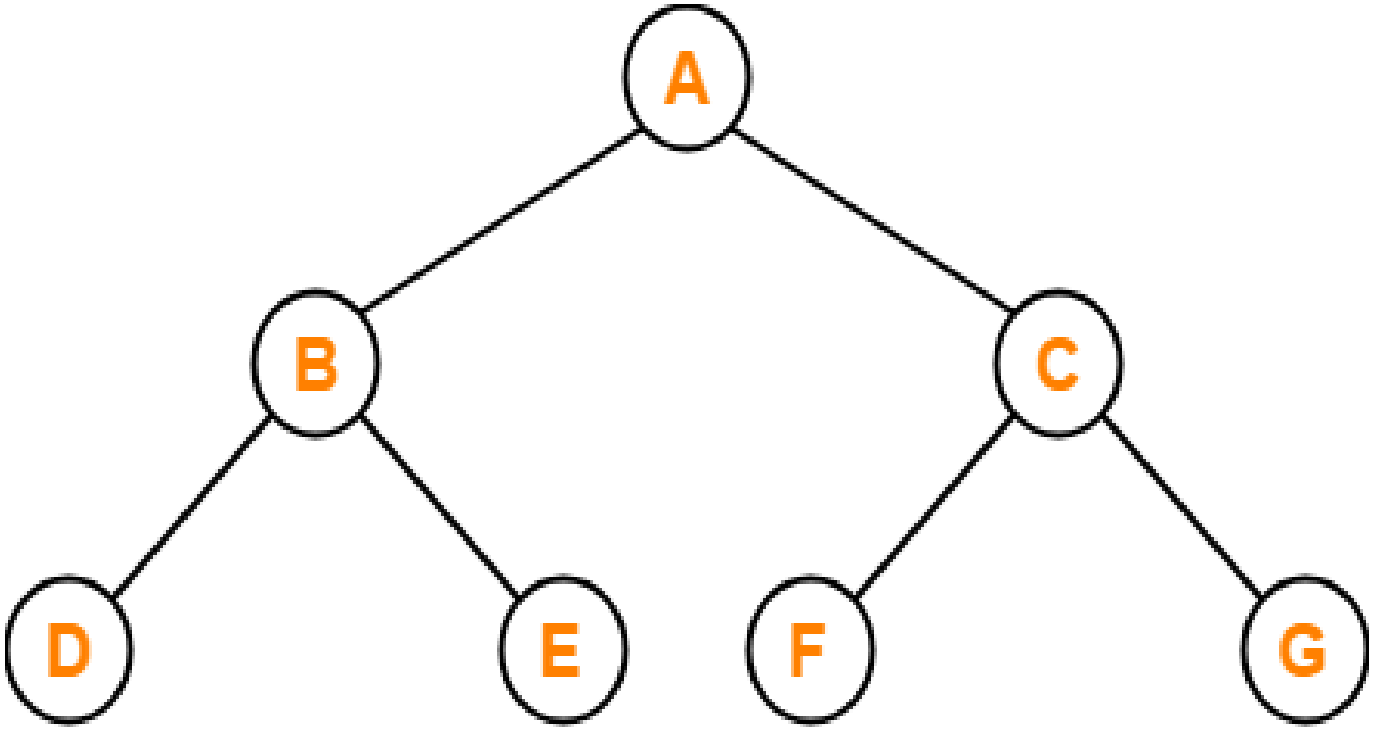
Postorder: **D, E, B, F, G, C, A**

Inorder: **D, B, E, A, F, C, G**

Postorder: **D, E, B, F, G, C, A**

Inorder: **D, B, E, A, F, C, G**

Ans:



Problem

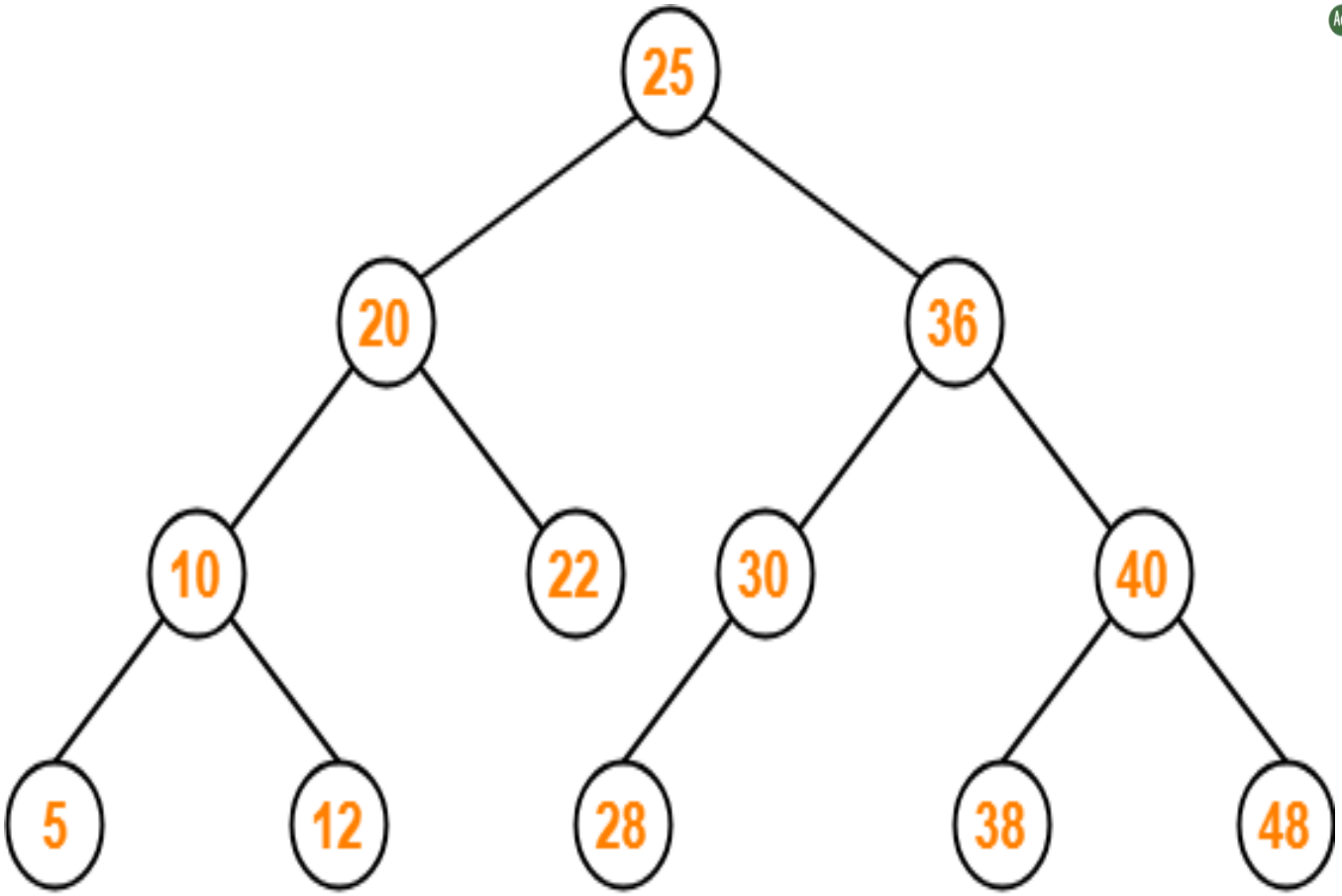
- Construct the BT and write the postorder of the tree:

Inorder: **dbeafcg**

Preorder: **abdecfg**

Binary Search Tree

- Binary Search Tree is a special kind of binary tree in which nodes are arranged in a specific order.
- In a binary search tree (BST), each node contains-
  - Only smaller values in its left sub tree.
  - Only larger values in its right sub tree.
  - Left and Right subtree must also be binary search tree.
  - There must be no duplicate nodes i.e two node can not have same value.



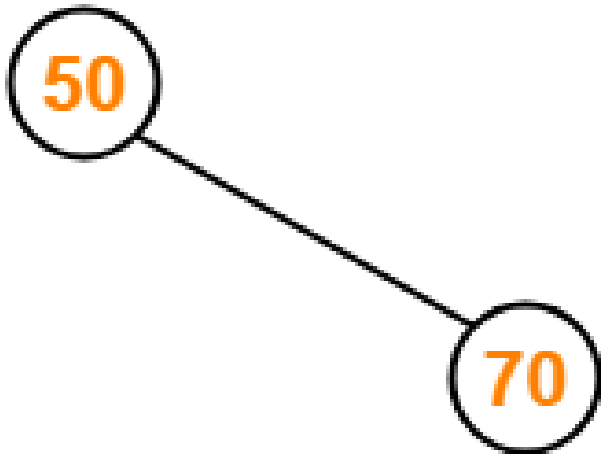
Binary Search Tree

Binary Search Tree Construction

- Construct a Binary Search Tree (BST) for the following sequence of numbers-
- 50, 70, 60, 20, 90, 10, 40, 100

Insert 50

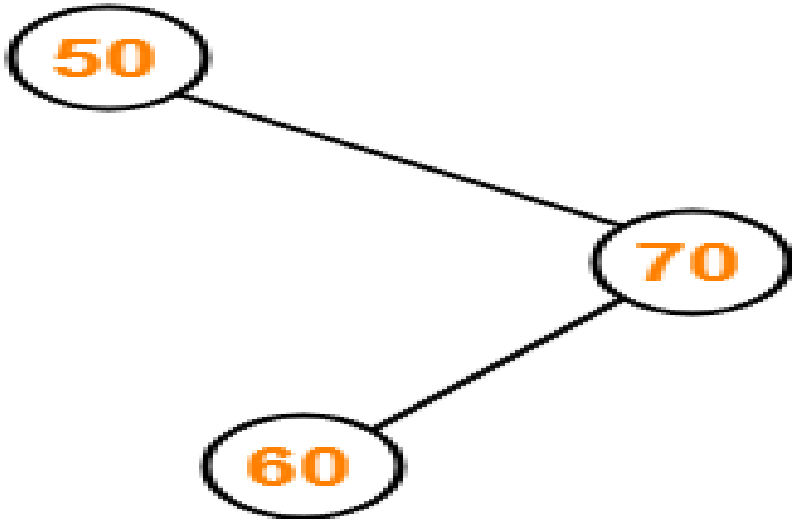
- **Insert 70**
- As 70 > 50, so insert 70 to the right of 50.



Cont…

Insert 60

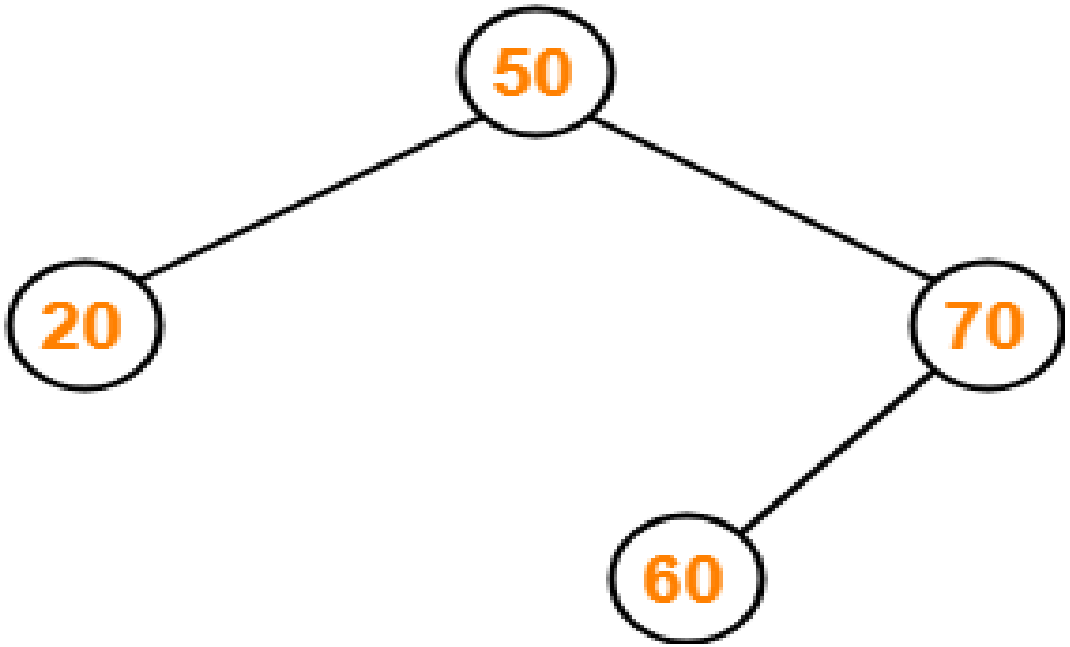
- As 60 > 50, so insert 60 to the right of 50.
- As 60 < 70, so insert 60 to the left of 70.



Cont…

Insert 20

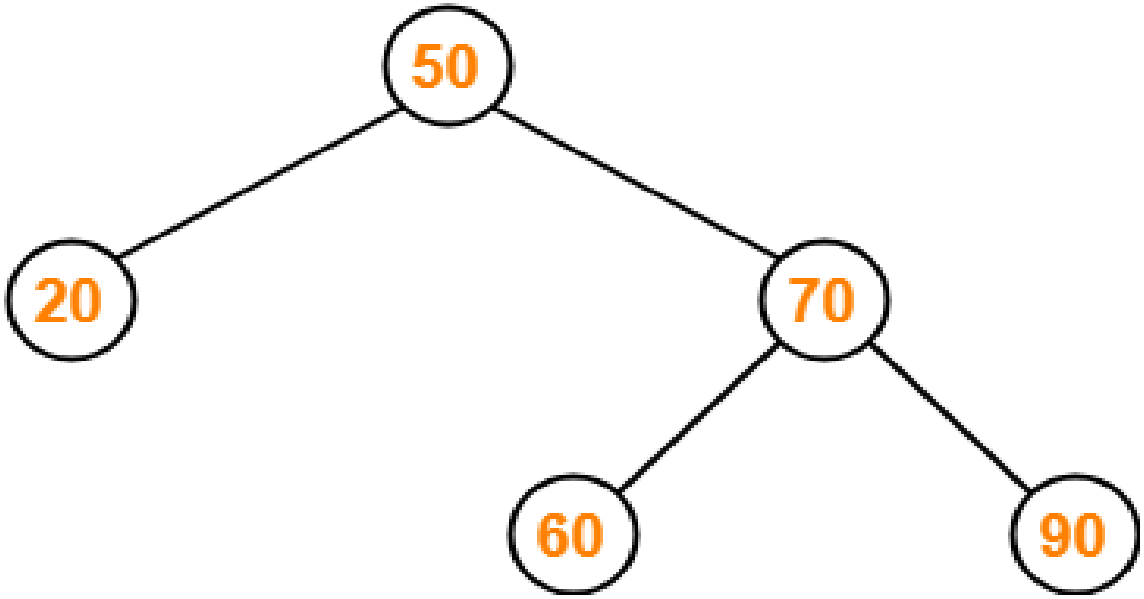
- As 20 < 50, so insert 20 to the left of 50.



Cont…

Insert 90

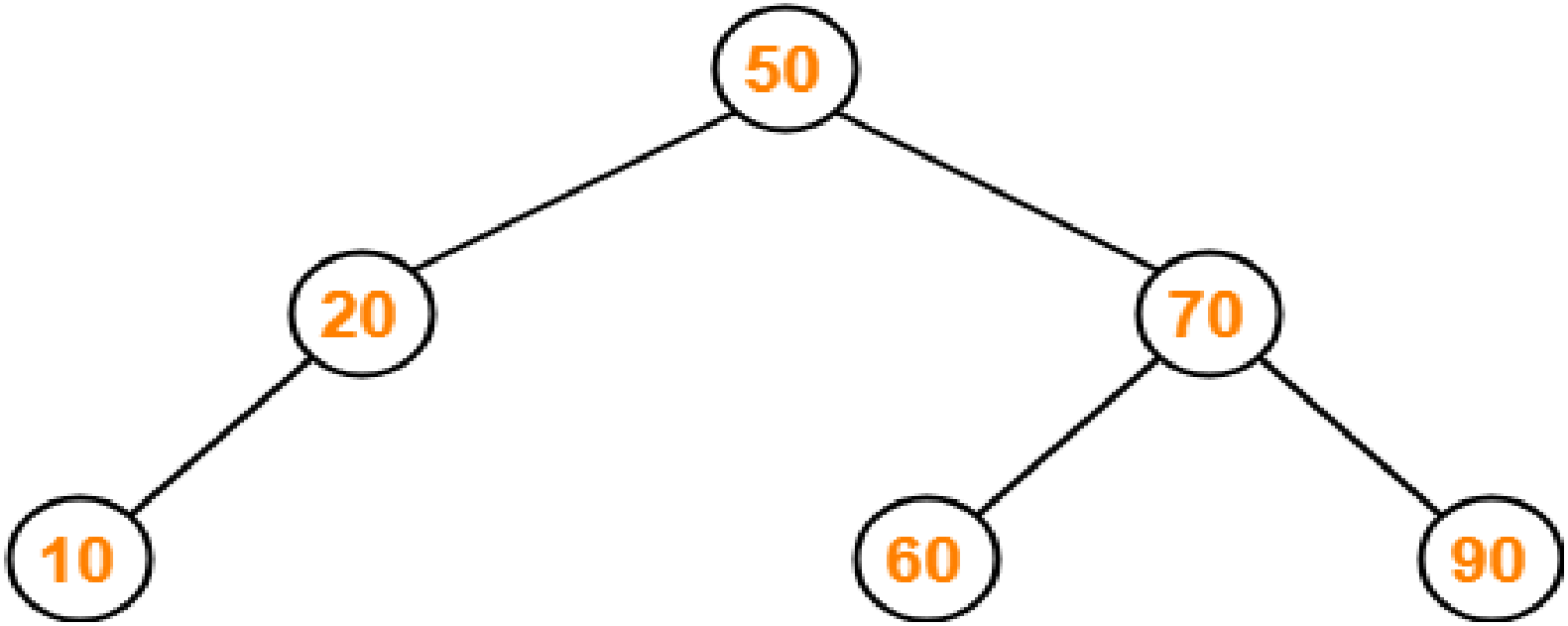
- As 90 > 50, so insert 90 to the right of 50.
- As 90 > 70, so insert 90 to the right of 70



Cont…

Insert 10

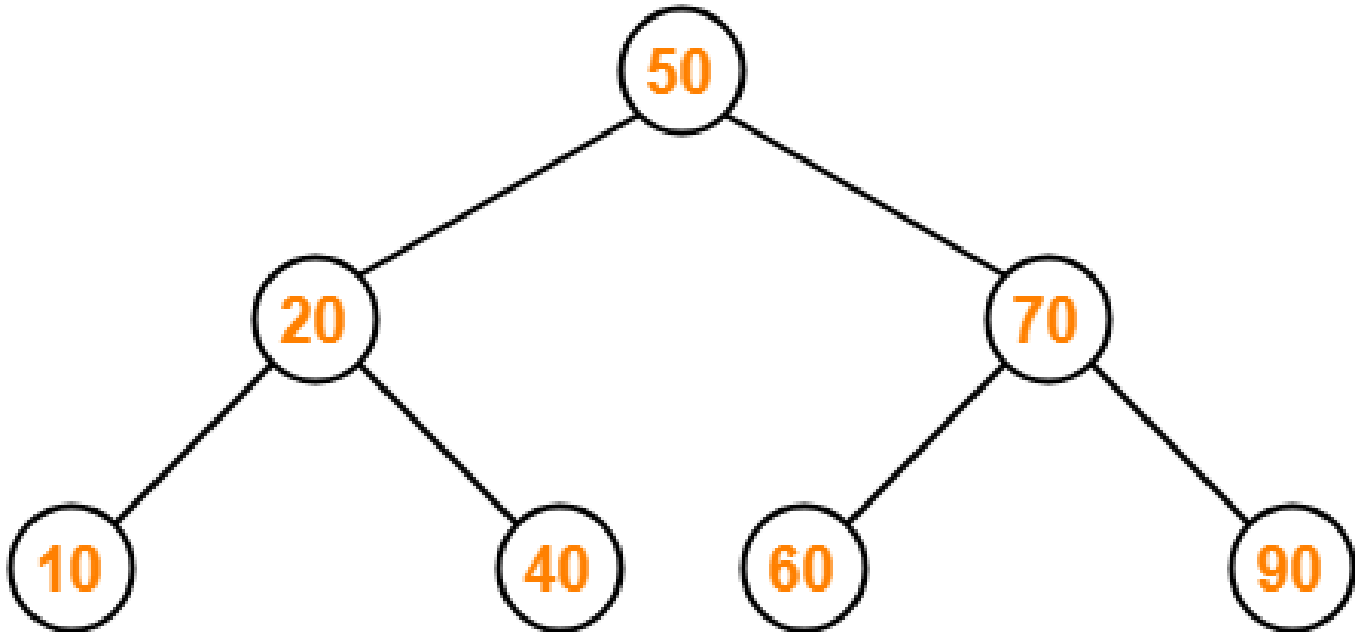
- As 10 < 50, so insert 10 to the left of 50.
- As 10 < 20, so insert 10 to the left of 20



Cont…

Insert 40

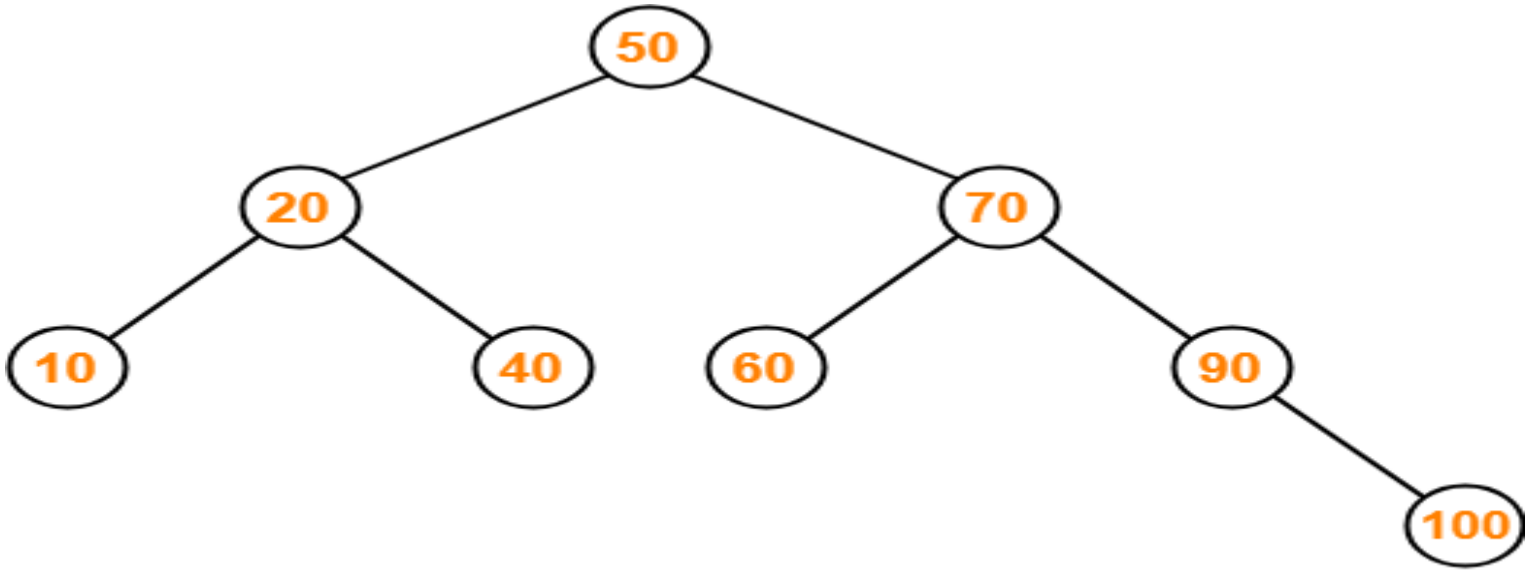
- As 40 < 50, so insert 40 to the left of 50.
- As 40 > 20, so insert 40 to the right of 20



Cont…

Insert 100

- As 100 > 50, so insert 100 to the right of 50.
- As 100 > 70, so insert 100 to the right of 70.
- As 100 > 90, so insert 100 to the right of 90.



Binary Search Tree

Problem-01

- A binary search tree is generated by inserting in order of the following integers-

50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24

- The number of nodes in the left subtree and right subtree of the root respectively is _____.

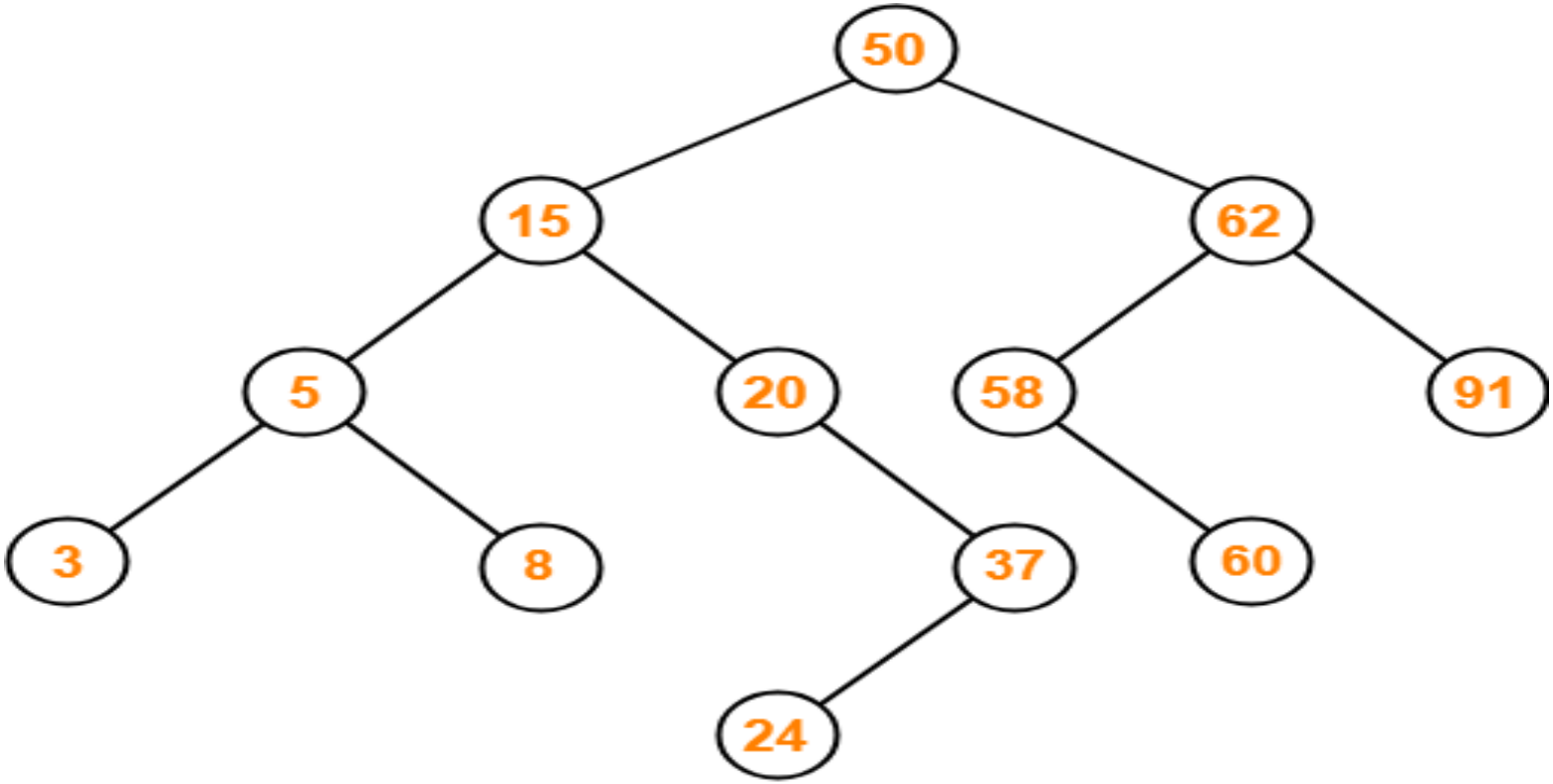
A. (4, 7)

B. (7, 4)

C. (8, 3)

D. (3, 8)

Number of nodes in the left subtree of the root = 7
Number of nodes in the right subtree of the root = 4



Binary Search Tree

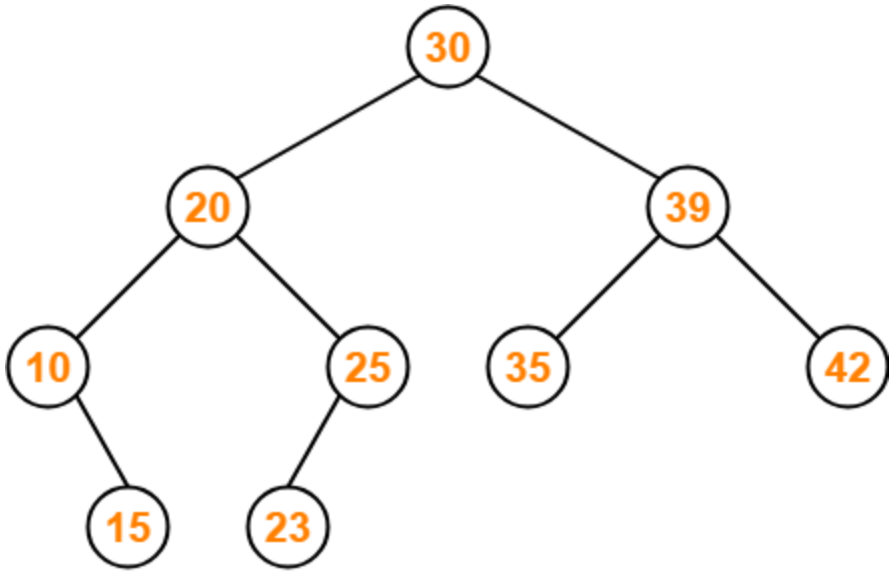
Binary Search Tree

Preorder Traversal :

30 , 20 , 10 , 15 , 25 , 23 , 39 , 35 , 42

Inorder Traversal :

10 , 15 , 20 , 23 , 25 , 30 , 35 , 39 , 42



Binary Search Tree

Binary Search Tree Operations

- Search Operation
- Insertion Operation
- Deletion Operation

🌳 Binary Search Tree (BST) Creation Algorithm

Input:
A list or sequence of integers (or comparable elements) to be inserted into the BST.

Example: [50, 30, 70, 20, 40, 60, 80]

Output:
A Binary Search Tree (BST) with all nodes inserted in the correct order.

Algorithm CreateBST(array)
Input: array of elements to insert into BST
Output: root node of the Binary Search Tree

1. Initialize root = NULL
2. For each element value in array:
    a. root = InsertNode(root, value)
3. Return root

Procedure InsertNode(node, value)
Input: node (current root), value (to insert)
Output: updated node

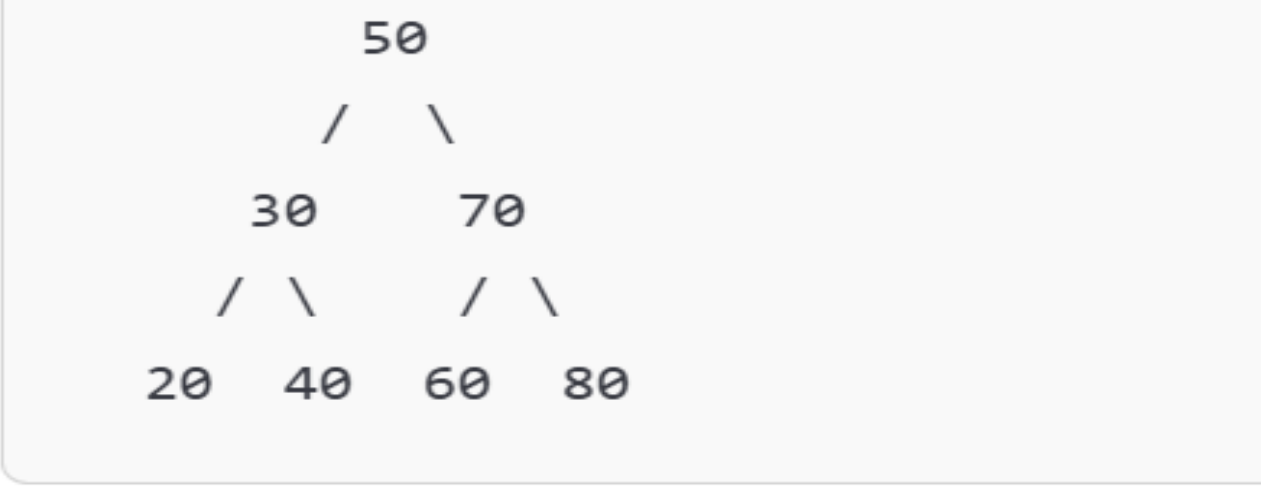
1. If node is NULL:
    a. Create a new node with value
    b. Return new node
2. If value < node.data:
    a. node.left = InsertNode(node.left, value)
3. Else if value > node.data:
    a. node.right = InsertNode(node.right, value)
4. Return node

🌟 Example Trace:

Input: `[50, 30, 70, 20, 40, 60, 80]`

Resulting BST:

plaintext



🔍 Algorithm to Search a Node in a Binary Search Tree (BST)

📥 Input:

- A **Binary Search Tree** (BST) with its `root` node.
- A **key** (integer or comparable value) to search for.

📤 Output:

- If the key is found: return the **node containing the key**.
- If the key is not found: return **NULL** (or "Not Found").

Algorithm SearchBST(root, key)

Input: root (node), key (value to search)
Output: node if key is found, otherwise NULL

1. Set current = root
2. While current is not NULL:
    a. If key == current.data:
        return current
    b. Else if key < current.data:
        current = current.left
    c. Else:
        current = current.right
3. Return NULL

1. Search Operation

- Search Operation is performed to search a particular element in the Binary Search Tree.

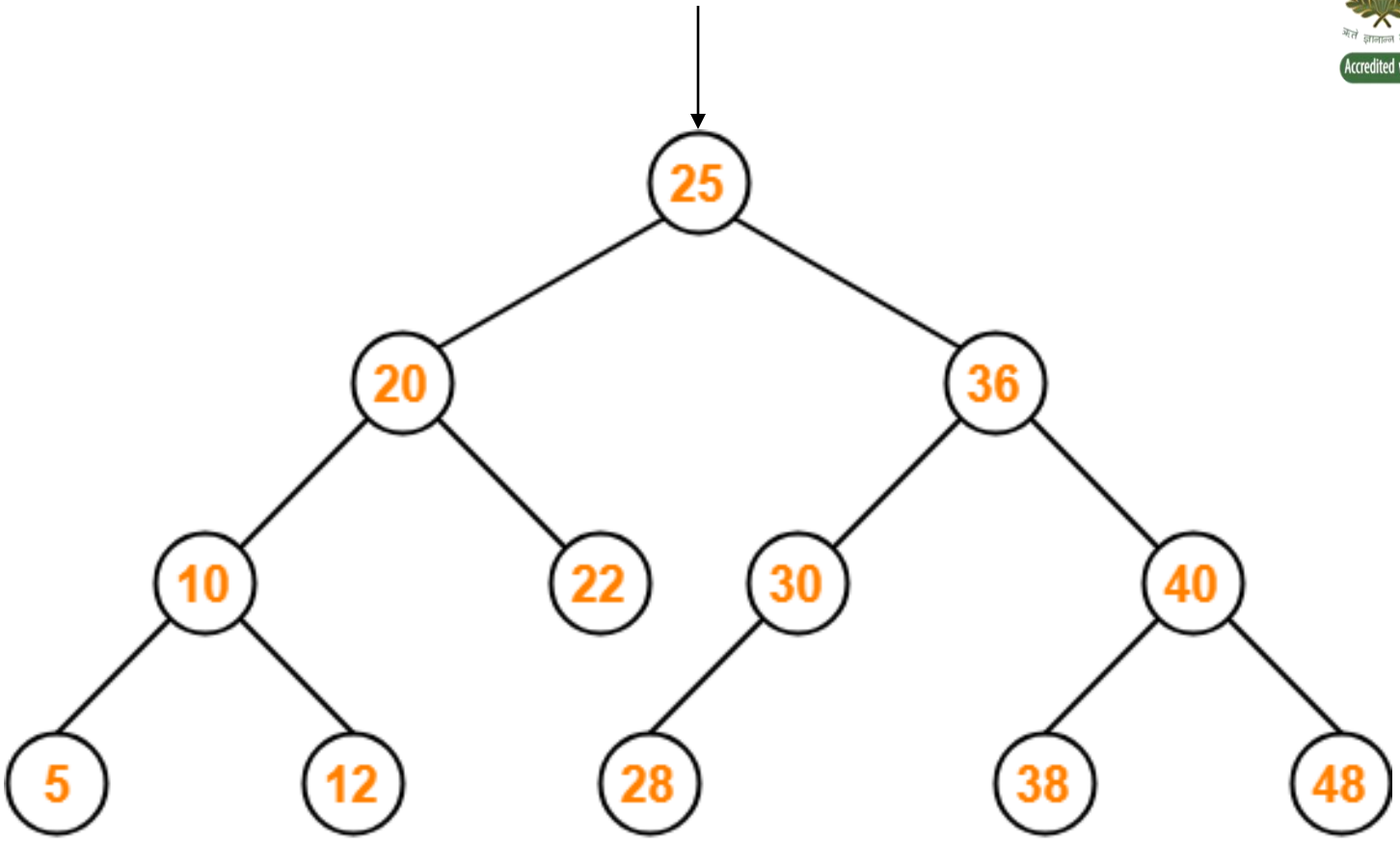
Rules

- For searching a given key (value) in the BST,
- Compare the key (value) with the value of root node.
- If the key (value) is present at the root node, then return the root node.

Cont…

- If the key is greater than the root node value, then search for the key (value) in the right subtree of root node.

- If the key is smaller than the root node value, then search for the key (value) in left subtree of root node.



Binary Search Tree

2. Insertion Operation

Rules

- Whenever an element (node) is to be inserted, first locate its proper location.

- Start searching from the root node.

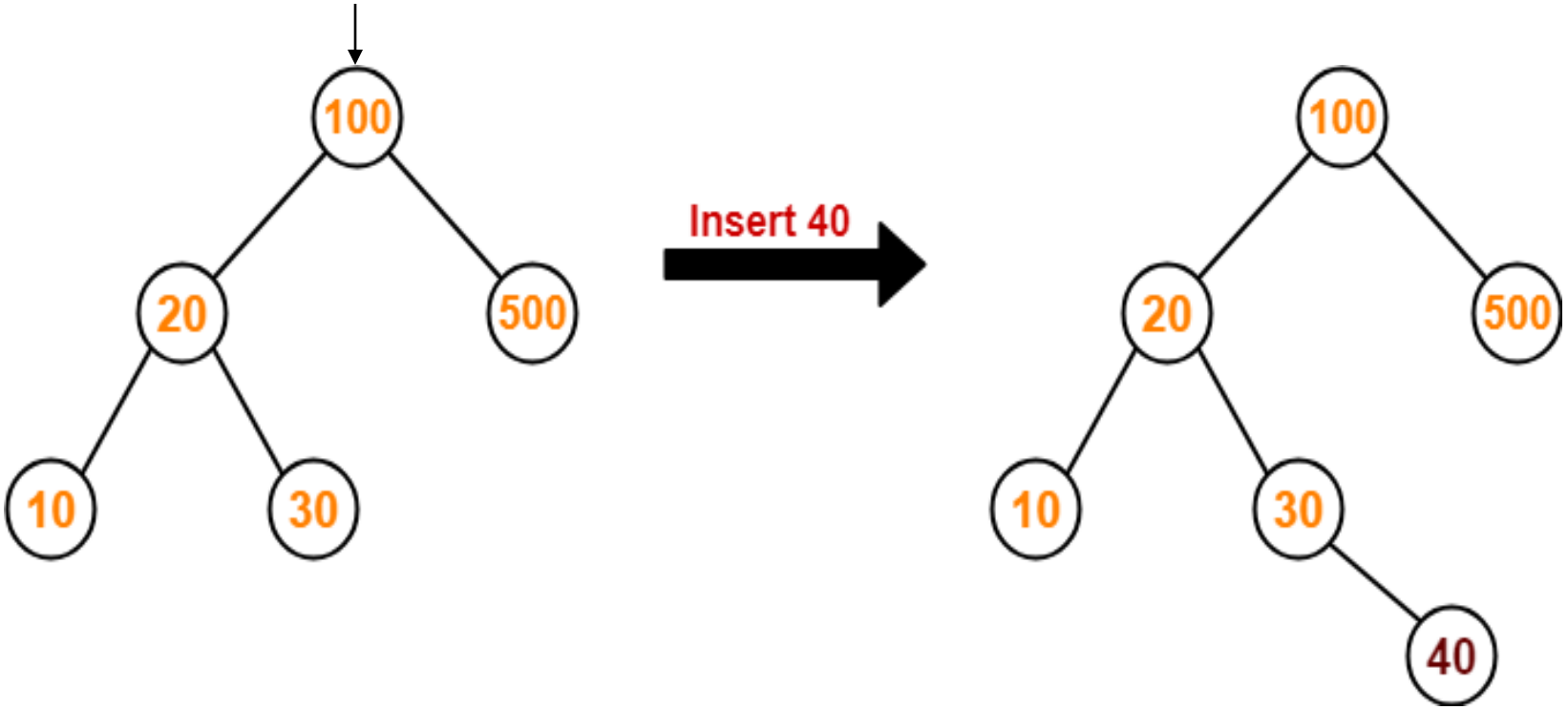
- If new element (node) is equal to the value of root node then node will not be insert because duplicate node is not allowed in BST.

- if the new element is less than value of root node , search for the empty location in the left subtree and insert the data.

- Otherwise, search for the empty location in the right subtree and insert the data.

Cont…

- Consider the following example where key = 40 is inserted in the given BST



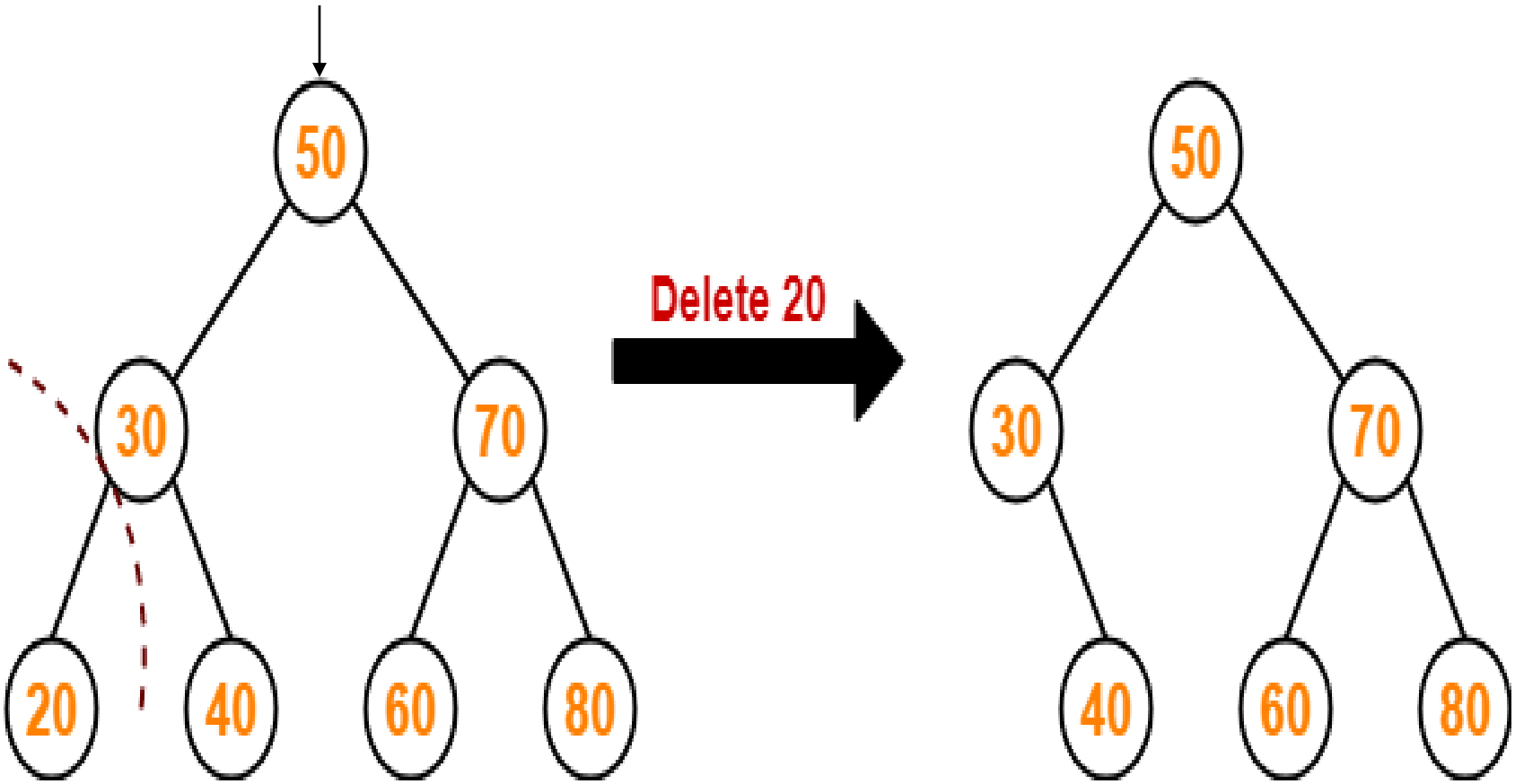
3. Deletion Operation

Case-01: Deletion Of A Node Having No Child (Leaf Node)

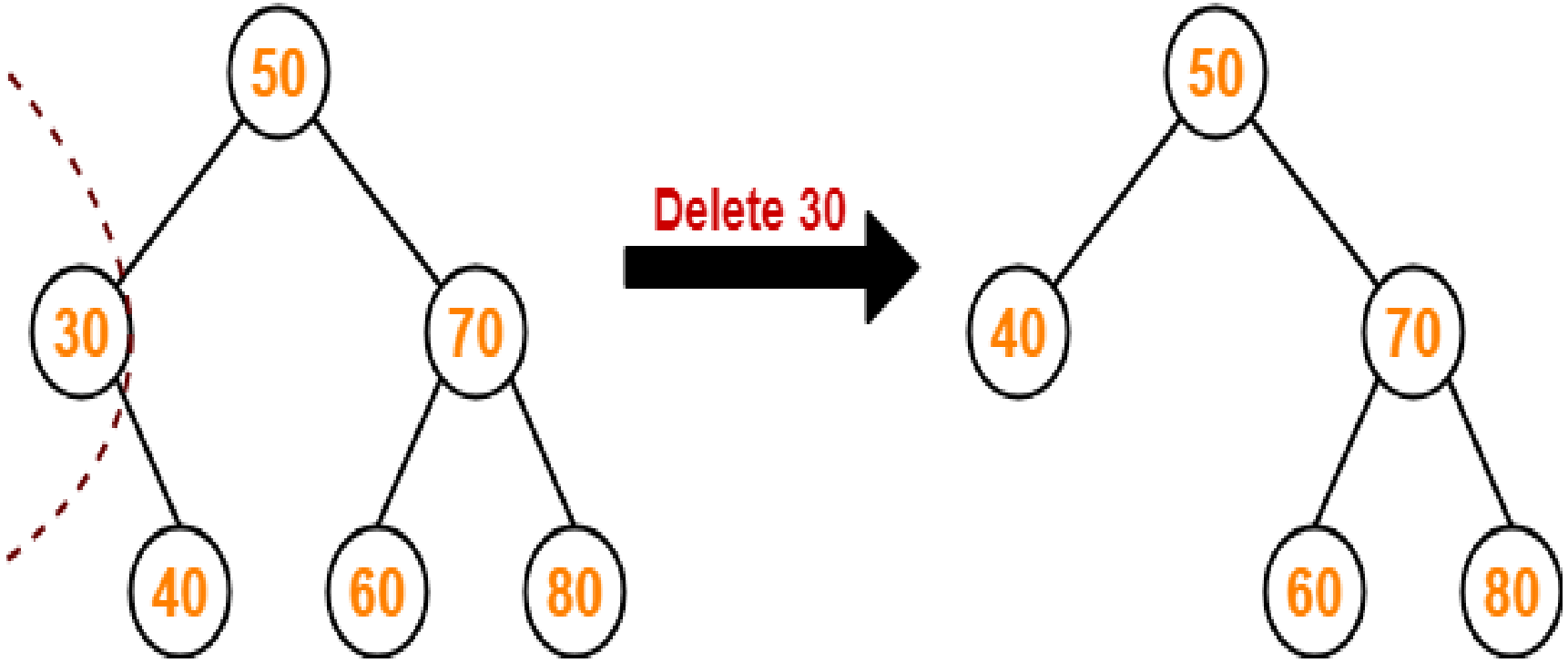
Case-02: Deletion Of A Node Having Only One Child

Case-03: Deletion Of A Node Having Two Children

Case-01: Deletion Of A Node Having No Child (Leaf Node)



Case-02: Deletion Of A Node Having Only One Child

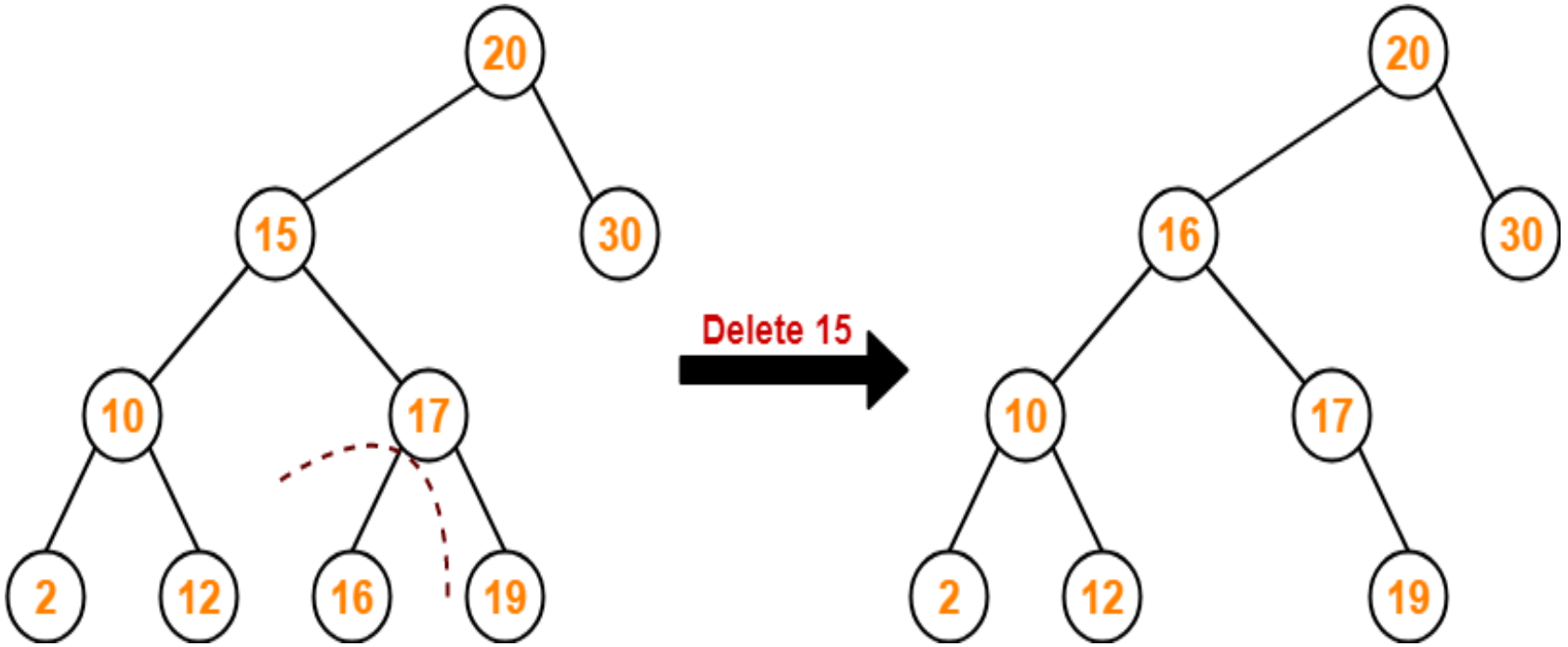


Case-03: Deletion Of A Node Having Two Children

- First delete the **inorder successor** of the **element to be deleted**.

- Using Case 1 or Case 2

- Then replace the **element to be deleted** with its **inorder successor**



Step:1 Inorder: 2,10,12,15,16,17,19,20,30
Inorder Successor of 15 is 16.

Step: 2 Delete the inorder successor of 15 is 16 by using Case-1

Step: 3 Replace the element to be deleted (15) with its inorder successor 16.

Binary Tree Representations

- A binary tree data structure is represented in a memory using two methods.
  - **Array Representation**
  - **Linked List Representation**

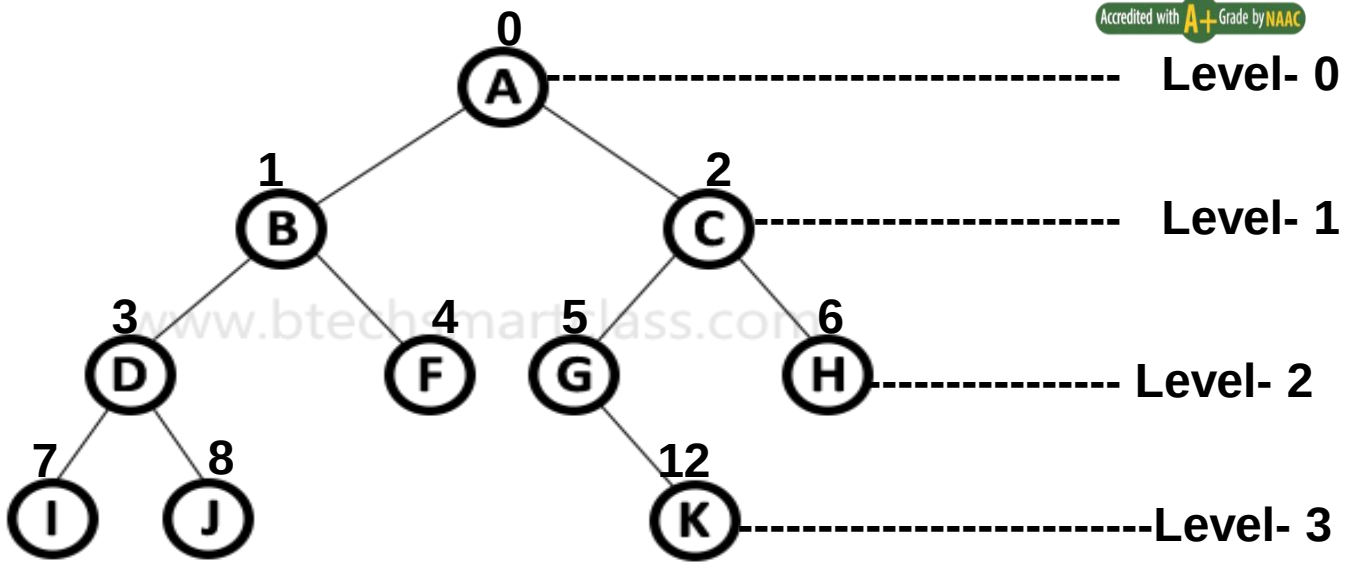
1. Array Representation of Binary Tree

- Use one-dimensional array (1-D Array) to represent a binary tree.

- For binary tree of height **‘h'** using array representation, we need one dimensional array with a maximum size of $\mathbf{2^{h+1} - 1}$.

- The root node is always at index 0.

Example:



Height of Binary Tree = h = 3

Total size of an array = 2^{h+1} -1 = 2^{3+1} – 1 = 2^4-1 = 16 – 1 = 15

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
A	B	C	D	F	G	H	I	J				K		

How to find out the index of a node

(1) parent(i) = Index of a parent node of any node having index i.

parent(i) = floor((i-1)/2) if i!=0

if i=0, then it is the root node and has no parent node.

Example:

parent(2)==floor(1/2)=floor(0.5)= 0

Cont…

(2) lchild(i) = Index of left child node of any node having index i.

lchild(i) = 2i+1

Example:

lchild(2)=2*2 + 1 = 4 + 1 = 5

Cont…

(3) rchild(i) = Index of right child node of any node having index i.

rchild(i) = 2i+2

Example:

rchild(2)=2*2 + 2 = 4 + 2 = 6

Cont…

(4) siblings:

- If left child at index i is given then its right sibling is at (i+1) index.

- If right child at index i is given then its left sibling is at (i - 1) index.

Example: 1

rightsibling(i)= Index of right sibling of left child having index i.

rightsibling(i)= i + 1

rightsibling(3)= 3 + 1 = 4

Example: 2

leftsibling(i)= Index of left sibling of right child having index i.

leftsibling(i)= i -1

leftsibling(4)= 4 - 1 = 3

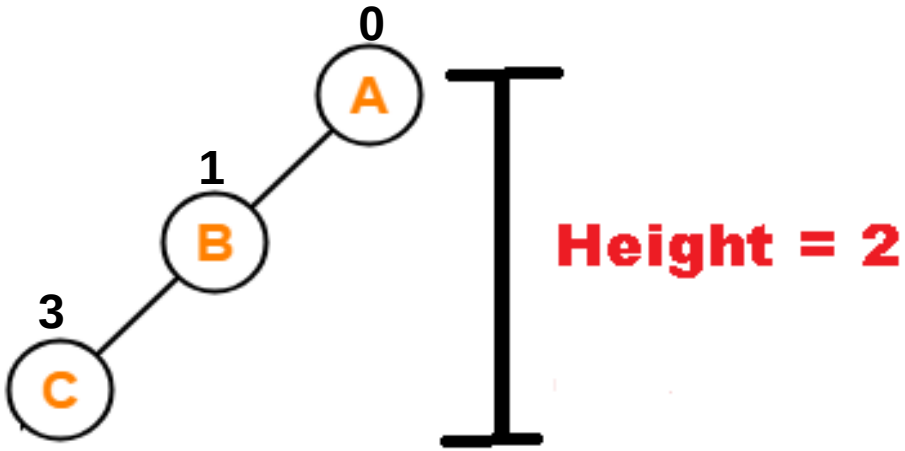
Cont…

Note:

- Array representation is more ideal for perfect binary tree.

- It is not suitable for other than perfect binary tree because it results in unnecessary wastage of memory.

Example: Left Skewed Binary tree.



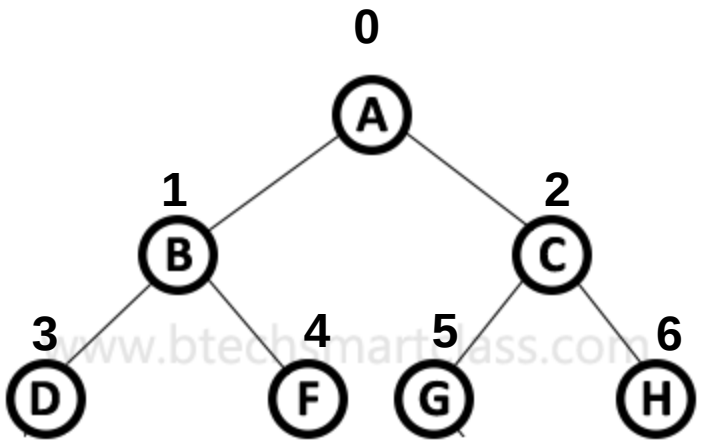
Height (h) 2

Total size of an array = $2^{h+1}-1 = 2^{2+1} -1 = 2^3 -1 = 8-1 = 7$

0	1	2	3	4	5	6
A	B		C			

Array Representation Of Binary Tree

Example 2: Perfect Binary Tree



Height = 2

Height (h) 2

Total size of an array = $2^{h+1}-1 = 2^{2+1}-1 = 2^3-1 = 8-1 = 7$

0	1	2	3	4	5	6
A	B	C	D	F	G	H

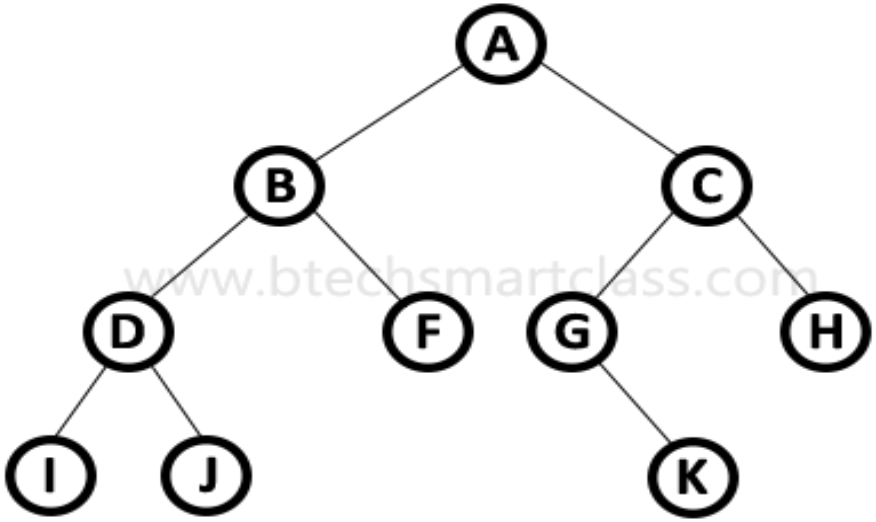
Array Representation Of Binary Tree

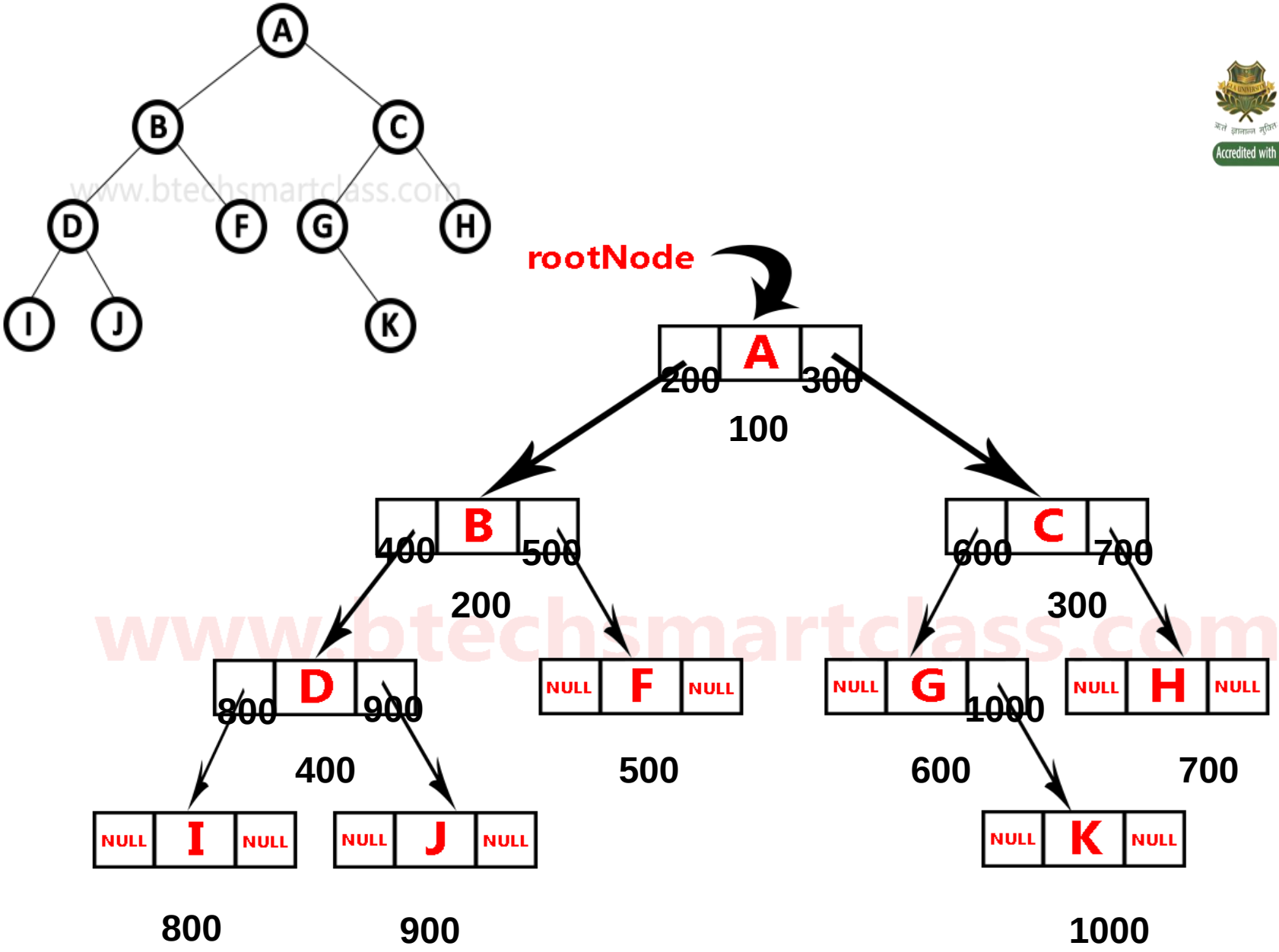
2. Linked List Representation of Binary Tree

- **We use a double linked list to represent a binary tree.**
- **In a double linked list, every node consists of three fields.**
  - **First field for storing left child address,**
  - **second for storing actual data and**
  - **third for storing right child address.**

Cont…

Example:





Linked List Representation Of Binary Tree